

UNIVERSIDADE DE SÃO PAULO
INSTITUTO DE FÍSICA DE SÃO CARLOS
BACHARELADO EM FÍSICA

CAIRO HENRIQUE VAZ COTRIM
No USP: 16814178

PROJETO 2 – SISTEMAS ALEATÓRIOS

7600017 – INTRODUÇÃO À FÍSICA COMPUTACIONAL

SÃO CARLOS – SP

2026

Sumário

1	Problema 1: Números Aleatórios, Integrais e Amostragem	2
1.1	(a)	2
1.1.1	Descrição do Problema	2
1.1.2	Estratégia de Solução	3
1.1.3	Resultados Obtidos	5
1.2	(b)	7
1.2.1	Descrição do Problema	7
1.2.2	Estratégia de Solução	7
1.2.3	Resultados Obtidos	11
1.3	(c)	13
1.3.1	Descrição do Problema	13
1.3.2	Estratégia de Solução	14
1.3.3	Resultados Obtidos	15
1.4	(d)	17
1.4.1	Descrição do Problema	17
1.4.2	Estratégia de Solução	17
1.4.3	Resultados Obtidos	20
2	Problema 2: Decaimento Radioativo	22
2.1	(a)	22
2.1.1	Descrição do Problema	22
2.1.2	Estratégia de Solução	22
2.1.3	Resultados Obtidos	25
2.2	(b)	27
2.2.1	Descrição do Problema	27
2.2.2	Estratégia de Solução	27
2.2.3	Resultados Obtidos	32
2.3	(c)	34
2.3.1	Descrição do Problema	34
2.3.2	Estratégia de Solução	35
2.3.3	Resultados Obtidos	38
3	Problema 3: Passeio Aleatório	39
3.1	(a)	39
3.1.1	Descrição do Problema	39
3.1.2	Estratégia de Solução	40
3.1.3	Resultados Obtidos	45
3.2	(b)	49
3.2.1	Descrição do Problema	49
3.2.2	Estratégia de Solução	49
3.2.3	Resultados Obtidos	53
3.3	(c) – Opcional	55
3.3.1	Descrição do Problema	55
3.3.2	Estratégia de Solução	56
3.3.3	Resultados Obtidos	56

Introdução

Este documento apresenta o relatório do Projeto 2, cujo principal objetivo é investigar sistemas aleatórios por meio de simulações computacionais utilizando a linguagem FORTRAN-77 e os recursos adicionais fornecidos pelo compilador GFortran.

Inicialmente, são estudadas propriedades de sequências de números pseudoaleatórios, incluindo suas médias, desvios padrão e a distribuição das médias de diferentes sequências. Em seguida, os números aleatórios são utilizados para estimar o valor de π e calcular integrais pelo método de Monte Carlo.

Também são investigados processos estocásticos, como o decaimento radioativo e o passeio aleatório em uma e duas dimensões. Para cada problema, são apresentados a descrição do fenômeno físico ou matemático envolvido, a estratégia computacional desenvolvida e os resultados obtidos das execuções próprias.

1 Problema 1: Números Aleatórios, Integrais e Amostragem

1.1 (a)

1.1.1 Descrição do Problema

O objetivo deste item é investigar numericamente as propriedades de sequências de números pseudoaleatórios geradas pela função intrínseca ‘rand()’ do FORTRAN.

Devem ser geradas

$$k = 10^2$$

sequências de números aleatórios de tamanho

$$N = 3^m, \quad m = 1, 2, \dots, 9.$$

Para cada sequência, devem ser calculados a média

$$\langle r \rangle$$

e o desvio padrão

$$\sigma_r = \sqrt{\langle r^2 \rangle - \langle r \rangle^2}.$$

Os resultados devem ser apresentados graficamente como função de N e comparados com os valores esperados para uma distribuição uniforme no intervalo

$$0 \leq r < 1.$$

Para essa distribuição, espera-se que

$$\langle r \rangle = \frac{1}{2}$$

e

$$\sigma_r = \frac{1}{2\sqrt{3}} \approx 0.288675.$$

1.1.2 Estratégia de Solução

A implementação foi realizada em FORTRAN utilizando a função intrínseca ‘rand()’ para a geração dos números pseudoaleatórios. Inicialmente, foi definida uma semente fixa, dada por

$$\text{iseed} = 42,$$

por meio da chamada ‘srand()’. Dessa forma, garante-se a reprodutibilidade dos resultados obtidos durante a execução do programa.

Para cada valor de

$$m = 1, 2, \dots, 9,$$

foi definido o tamanho da sequência como

$$N = 3^m.$$

Para cada um desses valores, foram geradas $k = 100$ sequências independentes. A sub-rotina ‘gerar_sequencia()’ é responsável por preencher o vetor ‘sequencia’ com N números pseudoaleatórios gerados por ‘rand()’.

Após a geração de cada sequência, sua média aritmética é calculada pela função ‘media()’, que implementa diretamente a expressão

$$\langle r \rangle = \frac{1}{N} \sum_{i=1}^N r_i.$$

O desvio padrão é calculado pela função ‘desvio_padrao()’. Inicialmente, é construído o vetor contendo os quadrados dos elementos da sequência, permitindo o cálculo de

$$\langle r^2 \rangle.$$

Em seguida, utiliza-se a relação

$$\sigma_r = \sqrt{\langle r^2 \rangle - \langle r \rangle^2}.$$

Para cada uma das 100 realizações associadas a um determinado valor de N , os valores calculados da média e do desvio padrão são armazenados, respectivamente, nos arquivos ‘tarefa-1a-saida-1.dat’ e ‘tarefa-1a-saida-2.dat’. Esses arquivos foram posteriormente utilizados para a construção dos gráficos apresentados a seguir.

```

1      program Tarefa1a
2
3      character(50) arquivo_media, arquivo_desvio_padrao
4
5      integer*4 k, m, N
6      parameter (k = 100)
7
8      real*8 media, media_sequencia
```

```

9      real*8 desvio_padrao, desvio_padrao_sequencia
10     real*8 sequencia(3**9)
11
12     integer*4 iseed
13     parameter (iseed = 42)
14     call srand(iseed)
15
16     arquivo_media = 'tarefa-1a-saida-1.dat'
17     arquivo_desvio_padrao = 'tarefa-1a-saida-2.dat'
18
19     open(unit=10, file=arquivo_media)
20     open(unit=20, file=arquivo_desvio_padrao)
21
22     ! Para cada  $N = 3^m$ , gera k sequências e coloca os dados nos arquivos de
saída
23     do m = 1, 9
24         N = 3 ** m
25         do i = 1, k
26             call gerar_sequencia(sequencia, N)
27             media_sequencia = media(sequencia, N)
28             desvio_padrao_sequencia = desvio_padrao(sequencia, N)
29             print *, media_sequencia
30             print *, desvio_padrao_sequencia
31             write(10, *) N, media_sequencia
32             write(20, *) N, desvio_padrao_sequencia
33         end do
34     end do
35
36     close(10)
37     close(20)
38
39     end
40
41     subroutine gerar_sequencia(sequencia, tamanho)
42     ! Gera uma sequencia de numeros aleatorios uniformemente
43     ! Entrada: tamanho - numero de elementos da sequencia.
44     ! Saida: sequencia - vetor contendo os numeros aleatorios.
45
46     integer*4 tamanho
47     real*8 sequencia(3**9)
48
49     do i = 1, tamanho
50         sequencia(i) = rand()
51     end do
52
53     end
54
55     function media(sequencia, tamanho)
56     ! Calcula a media aritmetica dos elementos de uma sequencia.
57     ! Entrada: tamanho - numero de elementos da sequencia.
58     ! Saida: media - media aritmetica dos elementos da sequencia.

```

```

59
60     integer*4 tamanho
61     real*8 soma, media
62     real*8 sequencia(3**9)
63
64     soma = 0
65     do i = 1, tamanho
66         soma = soma + sequencia(i)
67     end do
68
69     media = soma / tamanho
70
71     return
72 end
73
74 function desvio_padrao(sequencia, tamanho)
75     ! Calcula o desvio padrao dos elementos de uma sequencia
76     ! utilizando a fórmula sigma = <x^2> - <x>^2.
77     ! Entrada: tamanho - numero de elementos da sequencia.
78     ! Saida: desvio_padrao - desvio padrao da sequencia.
79
80     integer*4 tamanho
81     real*8 media, desvio_padrao
82     real*8 media_sequencia, media_quadratica_sequencia
83     real*8 sequencia(3**9), sequencia_quadrada(3**9)
84     save sequencia_quadrada
85
86     do i = 1, tamanho
87         sequencia_quadrada(i) = sequencia(i)**2
88     end do
89
90     media_sequencia = media(sequencia, tamanho)
91
92     media_quadratica_sequencia = media(sequencia_quadrada, tamanho)
93
94     desvio_padrao =
95     &sqrt(media_quadratica_sequencia - media_sequencia**2)
96
97     return
98 end

```

1.1.3 Resultados Obtidos

A Figura 1 apresenta os valores da média calculados para as diferentes sequências geradas. Observa-se que os resultados se distribuem em torno do valor teórico

$$\langle r \rangle = \frac{1}{2}.$$

Além disso, à medida que o tamanho N das sequências aumenta, as flutuações dos valores obtidos em torno da média teórica tornam-se menores. Esse comportamento é

esperado, pois a média amostral tende ao valor esperado da distribuição conforme o número de elementos da amostra aumenta.

De maneira análoga, a Figura 2 apresenta os valores calculados para o desvio padrão das sequências. Os resultados aproximam-se do valor teórico

$$\sigma_r = \frac{1}{2\sqrt{3}} \approx 0.288675,$$

característico da distribuição uniforme no intervalo $[0, 1)$.

Para valores pequenos de N , observa-se uma dispersão maior nos valores calculados, consequência das flutuações estatísticas associadas ao número reduzido de elementos em cada sequência. Com o aumento de N , os valores do desvio padrão tornam-se progressivamente mais concentrados em torno do resultado teórico.

Assim, os resultados numéricos obtidos são consistentes com o comportamento esperado para uma distribuição uniforme. O aumento do tamanho das sequências reduz as flutuações estatísticas das estimativas da média e do desvio padrão, levando os resultados a uma concordância cada vez maior com os valores teóricos.

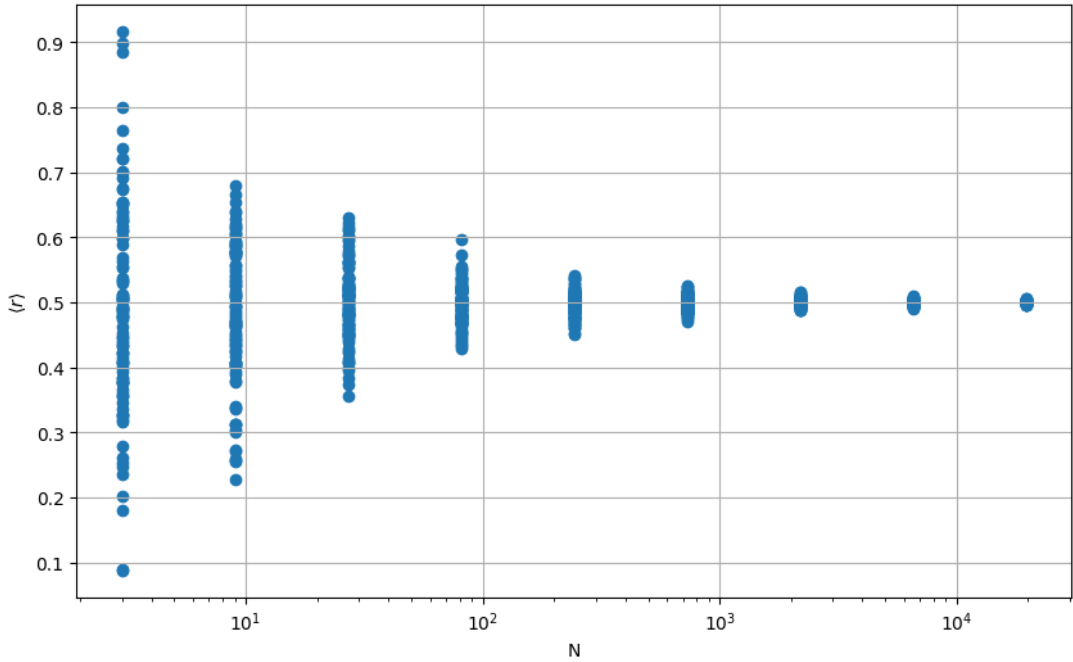


Figura 1: Valor médio $\langle r \rangle$ em função do tamanho da sequência N .

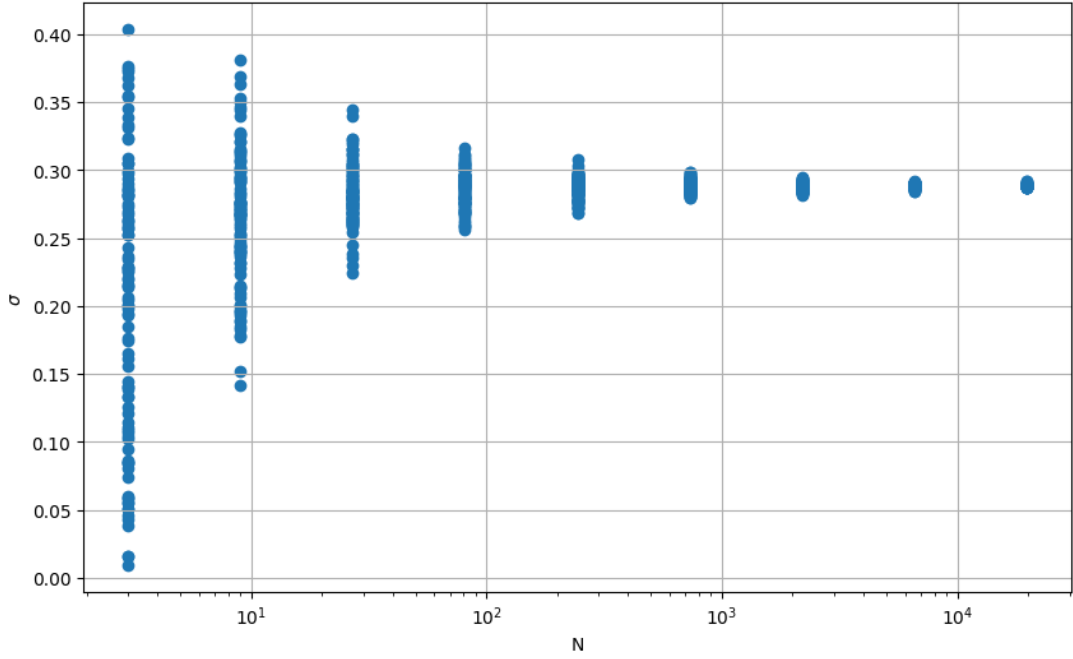


Figura 2: Desvio padrão σ_r em função do tamanho da sequência N .

1.2 (b)

1.2.1 Descrição do Problema

O objetivo é analisar a distribuição dos valores médios

$$\langle r \rangle$$

obtidos para diferentes sequências de números aleatórios.

Devem ser construídos histogramas normalizados dos valores médios para

$$N = 3^2, \quad N = 3^3, \quad N = 3^4.$$

Esses histogramas devem ser comparados com as distribuições Gaussianas esperadas a partir do Teorema Central do Limite.

Também deve ser analisado o desvio padrão das médias,

$$\sigma_{\langle r \rangle},$$

como função de N , verificando o comportamento esperado

$$\sigma_{\langle r \rangle} \sim \frac{1}{\sqrt{N}}.$$

1.2.2 Estratégia de Solução

A estratégia adotada consiste em gerar diversas sequências independentes de números pseudoaleatórios e analisar a distribuição de suas respectivas médias. Para cada sequência de tamanho N , a média é calculada por meio da função ‘media()’, que implementa a expressão

$$\langle r \rangle = \frac{1}{N} \sum_{i=1}^N r_i.$$

Inicialmente, foram considerados os tamanhos

$$N = 3^2, \quad N = 3^3, \quad N = 3^4.$$

Para cada um desses valores, foram geradas $k = 100$ sequências independentes. A média de cada sequência foi armazenada no arquivo ‘tarefa-1b-saida-1.dat’, juntamente com o respectivo valor de N . Esses dados foram posteriormente utilizados para a construção dos histogramas normalizados e para sua comparação com as distribuições Gaussianas previstas teoricamente.

De acordo com o Teorema Central do Limite, a distribuição das médias amostrais tende a uma distribuição Gaussiana à medida que o número de elementos de cada sequência aumenta. Como os números gerados por ‘rand()’ seguem uma distribuição uniforme no intervalo $[0, 1)$, a média teórica é

$$\mu_{\langle r \rangle} = \frac{1}{2}.$$

Além disso, o desvio padrão da média é dado por

$$\sigma_{\langle r \rangle} = \frac{\sigma_r}{\sqrt{N}} = \frac{1}{2\sqrt{3N}}.$$

Assim, para cada valor de N , a distribuição Gaussiana utilizada para comparação possui média

$$\mu = \frac{1}{2}$$

e desvio padrão

$$\sigma = \frac{1}{2\sqrt{3N}}.$$

Em uma segunda etapa, foi analisada a dependência do desvio padrão das médias com o tamanho das sequências. Para cada valor

$$N = 3^m, \quad m = 1, 2, \dots, 9,$$

foram geradas novamente $k = 100$ sequências independentes. As respectivas médias foram armazenadas no vetor ‘sequencia_medias’, e o desvio padrão desse conjunto de médias foi calculado pela função ‘desvio_padrao()’.

Os valores obtidos foram armazenados no arquivo ‘tarefa-1b-saida-2.dat’ e utilizados para construir o gráfico de

$$\sigma_{\langle r \rangle}$$

em função de N . A representação em escala log-log permite verificar diretamente o comportamento de lei de potência previsto,

$$\sigma_{\langle r \rangle} \propto N^{-1/2}.$$

```

1
2  program Tarefa1b
3
4  character(50) arquivo_media, arquivo_desvio_padrao
5
6  integer*4 k, m, N
7  parameter (k = 100)
8
9  real*8 media, media_sequencia
10 real*8 desvio_padrao, desvio_padrao_das_medias
11 real*8 sequencia(3**9), sequencia_medias(3**9)
12
13 integer*4 iseed
14 parameter (iseed = 42)
15 call srand(iseed)
16
17 arquivo_media = 'tarefa-1b-saida-1.dat'
18 arquivo_desvio_padrao = 'tarefa-1b-saida-2.dat'
19
20 open(unit=10, file=arquivo_media)
21
22 ! Gera 3 histogramas com k dados cada (dados de sequência)
23 do m = 2, 4
24     N = 3 ** m
25     do i = 1, k
26         call gerar_sequencia(sequencia, N)
27         media_sequencia = media(sequencia, N)
28         write(10, *) N, media_sequencia
29     end do
30 end do
31
32 close(10)
33
34 open(unit=20, file=arquivo_desvio_padrao)
35
36 ! Gera o gráfico de desvio padrão em função de N
37 do m = 1, 9
38     N = 3 ** m
39     do i = 1, k
40         call gerar_sequencia(sequencia, N)
41         media_sequencia = media(sequencia, N)
42         sequencia_medias(i) = media_sequencia
43     end do
44     desvio_padrao_das_medias =
45 &     desvio_padrao(sequencia_medias, k)
46     write(20, *) N, desvio_padrao_das_medias
47 end do
48
49 close(20)
50
51 end

```

```

52
53 subroutine gerar_sequencia(sequencia, tamanho)
54 ! Gera uma sequencia de numeros aleatorios uniformemente
55 ! Entrada: tamanho - numero de elementos da sequencia.
56 ! Saida: sequencia - vetor contendo os numeros aleatorios.
57
58 integer*4 tamanho
59 real*8 sequencia(3**9)
60
61 do i = 1, tamanho
62     sequencia(i) = rand()
63 end do
64
65 end
66
67 function media(sequencia, tamanho)
68 ! Calcula a media aritmetica dos elementos de uma sequencia.
69 ! Entrada: tamanho - numero de elementos da sequencia.
70 ! Saida: media - media aritmetica dos elementos da sequencia.
71
72 integer*4 tamanho
73 real*8 soma, media
74 real*8 sequencia(3**9)
75
76 soma = 0
77 do i = 1, tamanho
78     soma = soma + sequencia(i)
79 end do
80
81 media = soma / tamanho
82
83 return
84 end
85
86 function desvio_padrao(sequencia, tamanho)
87 ! Calcula o desvio padrao dos elementos de uma sequencia.
88 ! Entrada: tamanho - numero de elementos da sequencia.
89 ! Saida: desvio_padrao - desvio padrao da sequencia.
90
91 integer*4 tamanho
92 real*8 media, desvio_padrao
93 real*8 media_sequencia, media_quadratica_sequencia
94 real*8 sequencia(3**9), sequencia_quadrada(3**9)
95 save sequencia_quadrada
96
97 do i = 1, tamanho
98     sequencia_quadrada(i) = sequencia(i)**2
99 end do
100
101 media_sequencia = media(sequencia, tamanho)
102

```

```

103 media_quadratca_sequencia = media(sequencia_quadrada, tamanho)
104
105 desvio_padrao =
106 &sqrt(media_quadratca_sequencia - media_sequencia**2)
107
108 return
109 end

```

1.2.3 Resultados Obtidos

A Figura 3 apresenta os histogramas normalizados das médias obtidas para os três tamanhos de sequência considerados. Os resultados mostram que a distribuição das médias se concentra em torno do valor teórico

$$\langle r \rangle = \frac{1}{2}.$$

Observa-se que, mesmo para os valores relativamente pequenos de N considerados, a distribuição das médias apresenta um comportamento aproximadamente Gaussiano, em concordância com o Teorema Central do Limite. À medida que N aumenta, a dispersão dos valores médios em torno de $1/2$ diminui.

Esse comportamento pode ser compreendido a partir da expressão teórica para o desvio padrão das médias,

$$\sigma_{\langle r \rangle} = \frac{1}{2\sqrt{3N}}.$$

Portanto, embora a média das distribuições permaneça aproximadamente constante e igual a $1/2$, sua largura diminui com o aumento do número de elementos em cada sequência. Consequentemente, as médias calculadas tornam-se cada vez mais concentradas em torno do valor esperado.

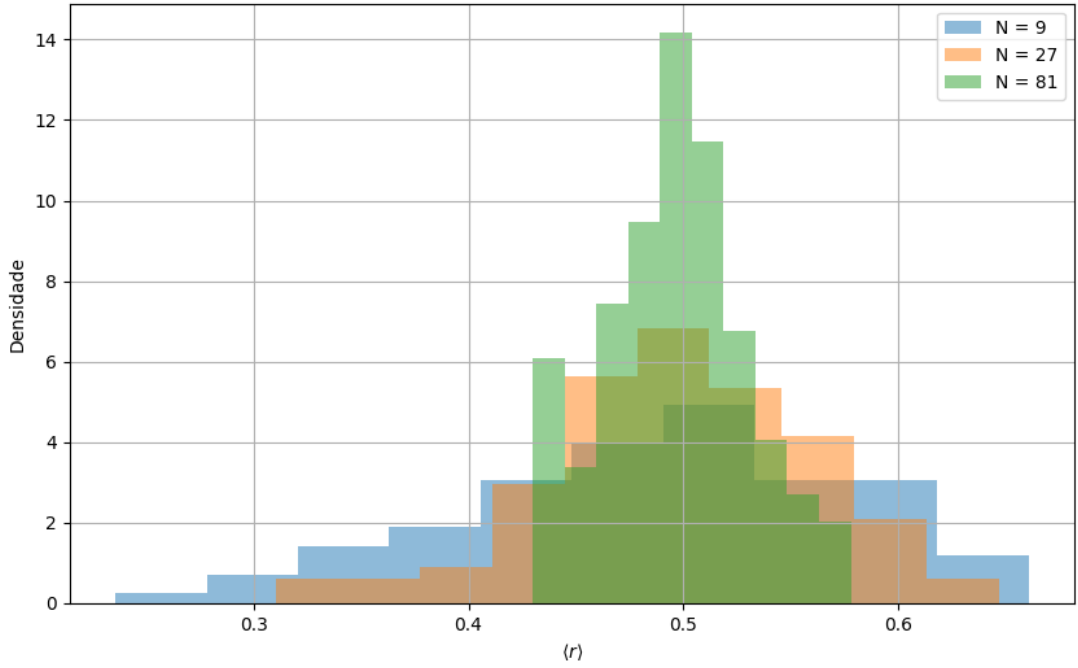


Figura 3: Histogramas normalizados dos valores médios $\langle r \rangle$ para diferentes valores de N .

A Figura 4 apresenta o desvio padrão das médias como função do tamanho N das seqüências em escala log-log. Os resultados numéricos seguem o comportamento decrescente previsto teoricamente,

$$\sigma_{\langle r \rangle} \propto \frac{1}{\sqrt{N}}.$$

A curva teórica utilizada para comparação é dada por

$$\sigma_{\langle r \rangle} = \frac{1}{2\sqrt{3N}} \approx \frac{0.288675}{\sqrt{N}}.$$

A concordância observada confirma a previsão do Teorema Central do Limite para a redução das flutuações da média amostral. Em particular, o gráfico em escala log-log evidencia o caráter de lei de potência da dependência entre o desvio padrão das médias e o tamanho da amostra, com expoente esperado igual a

$$-\frac{1}{2}.$$

Assim, tanto a aproximação Gaussiana observada nos histogramas quanto a dependência de

$$\sigma_{\langle r \rangle}$$

com N são consistentes com as previsões estatísticas para a média de variáveis aleatórias independentes e identicamente distribuídas.

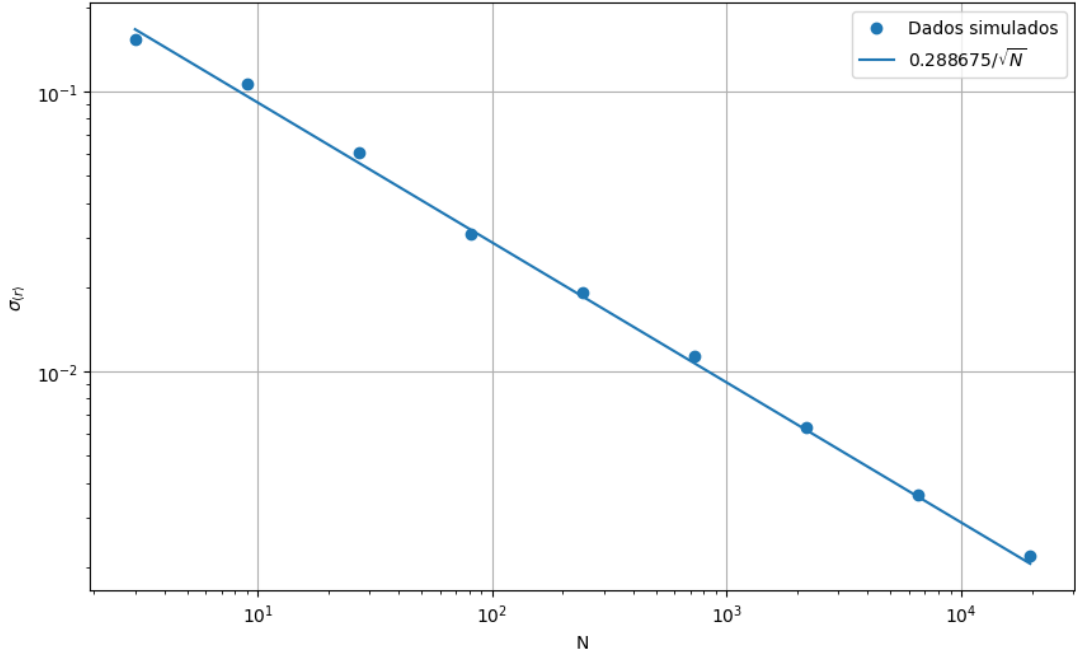


Figura 4: Desvio padrão das médias $\sigma_{\langle r \rangle}$ em função de N em escala log-log e sua comparação com a curva prevista pelo Teorema Central do Limite.

1.3 (c)

1.3.1 Descrição do Problema

Neste item, os números aleatórios são utilizados para estimar o valor de π pelo método de Monte Carlo.

O algoritmo consiste em gerar pares aleatórios

$$(x_i, y_i)$$

no quadrado definido por

$$-1 \leq x_i \leq 1, \quad -1 \leq y_i \leq 1.$$

Para cada ponto, verifica-se a condição

$$x_i^2 + y_i^2 \leq 1.$$

Caso a condição seja satisfeita, o ponto pertence ao círculo unitário. Como a área do quadrado é

$$A_{\text{quad}} = 4$$

e a área do círculo unitário é

$$A_{\text{circ}} = \pi,$$

a estimativa de π é dada por

$$\pi \approx 4 \frac{N_{\text{dentro}}}{N_{\text{total}}}.$$

O objetivo é determinar o valor de N_{total} necessário para obter uma estimativa de π com quatro casas decimais de precisão.

1.3.2 Estratégia de Solução

A estimativa foi realizada por meio do método de Monte Carlo, utilizando a razão entre o número de pontos localizados no interior do círculo unitário e o número total de pontos gerados no quadrado que o circunscreve.

Inicialmente, foi definida uma semente fixa,

$$\text{iseed} = 42,$$

de modo a tornar os resultados reprodutíveis. Para cada iteração, são gerados dois números aleatórios uniformemente distribuídos nos intervalos

$$-1 \leq x \leq 1$$

e

$$-1 \leq y \leq 1.$$

A função ‘numero_aleatorio_entre()’ realiza uma transformação linear dos valores produzidos por ‘rand()’, originalmente pertencentes ao intervalo

$$0 \leq r < 1,$$

para o intervalo desejado. Assim, os pares gerados são distribuídos uniformemente sobre a área do quadrado de lado igual a 2.

Para cada ponto gerado, é calculada a quantidade

$$x^2 + y^2.$$

Quando o resultado é menor que 1, o ponto é contabilizado como pertencente ao interior do círculo unitário. A estimativa acumulada de π é então atualizada a cada novo ponto por meio da expressão

$$\pi_{\text{MC}} = 4 \frac{N_{\text{dentro}}}{N_{\text{total}}}.$$

O programa continua gerando pontos até que o erro absoluto entre a estimativa e o valor de referência de π seja inferior à precisão estabelecida,

$$\epsilon = 10^{-4}.$$

A condição utilizada para interromper o algoritmo é, portanto,

$$|\pi_{\text{MC}} - \pi_{\text{real}}| < 10^{-4}.$$

```
1
2 program tarefa1c
3 ! Calcular pi utilizando o método de Monte Carlo em um círculo unitário.
4 ! Investigamos o numero N_total de pontos necessário para obter precisão de 4
  casas decimais.
5
```

```

6  integer*4 N_dentro, N_total
7  real*8 x, y, numero_aleatorio_entre, epsilon
8  parameter(epsilon = (10d0)**(-4)) ! precisão
9
10 real*8 pi_mc, pi_real
11 parameter(pi_real = 3.14159265358979323) ! valor real para calcular a precisão
12
13 integer*4 iseed
14 parameter(iseed = 42)
15 call srand(iseed)
16
17 ! loop para sortear N_total pontos até achar o valor bom de pi
18 N_dentro = 0
19 do N_total = 1, 10**8
20
21     x = numero_aleatorio_entre(-1d0, 1d0)
22     y = numero_aleatorio_entre(-1d0, 1d0)
23
24     if (x**2 + y**2 .LT. 1d0) then ! (x,y) pertencente ao círculo de raio 1
25         N_dentro = N_dentro + 1
26     end if
27
28     pi_mc = 4d0 * N_dentro / N_total ! A = pi r^2 = proporcao em relacao ao
    quadrado. Temos r = 1 e area do quadrado = 4
29
30     ! Se o erro for menor que epsilon, mostra os resultados finais
31     if (abs(pi_mc - pi_real) .LT. epsilon) then
32         print *, '0 numero de pontos necessario eh', N_total
33         print *, '0 valor de pi estimado eh', pi_mc
34         stop
35     end if
36
37 end do
38
39 end
40
41 function numero_aleatorio_entre(inicio, fim)
42 ! Gera um numero aleatorio uniformemente distribuido entre inicio e fim.
43 ! Entrada: inicio e fim - limites inferior e superior do intervalo.
44 ! Saida: numero_aleatorio_entre - numero aleatorio gerado no intervalo.
45 real*8 inicio, fim, numero_aleatorio_entre
46
47 numero_aleatorio_entre = (fim - inicio) * rand() + inicio ! bijecao linear
    entre o intervalo [0,1) e o intervalo [inicio, fim]
48
49 return
50 end

```

1.3.3 Resultados Obtidos

Saída:

O numero de pontos necessario eh 5116
O valor de pi estimado eh 3.1415168100078188

Geometricamente, o algoritmo pode ser interpretado como uma estimativa da razão entre as áreas de um círculo unitário e do quadrado que o circunscribe. Como os pontos são gerados uniformemente no interior do quadrado, a probabilidade de um ponto pertencer também ao círculo é igual à razão entre suas áreas,

$$P(\text{ponto dentro}) = \frac{A_{\text{circ}}}{A_{\text{quad}}} = \frac{\pi}{4}.$$

Consequentemente, para um número suficientemente grande de pontos, a frequência relativa de pontos localizados no círculo tende a essa probabilidade,

$$\frac{N_{\text{dentro}}}{N_{\text{total}}} \longrightarrow \frac{\pi}{4}.$$

Multiplicando essa relação por 4, obtém-se a estimativa utilizada pelo algoritmo.

Para a semente escolhida, o programa encontrou uma estimativa dentro da tolerância estabelecida após a geração de

$$N_{\text{total}} = 5116$$

pontos, obtendo

$$\pi_{\text{MC}} = 3.1415168100078188.$$

O erro absoluto em relação ao valor de referência é

$$|\pi_{\text{MC}} - \pi_{\text{real}}| \approx 7.58 \times 10^{-5},$$

valor inferior à tolerância especificada,

$$\epsilon = 10^{-4}.$$

Assim, o critério de precisão adotado foi satisfeito.

É importante observar, entretanto, que o valor

$$N_{\text{total}} = 5116$$

não representa uma quantidade universalmente necessária para garantir quatro casas decimais de precisão. Como o método é baseado em números pseudoaleatórios, a estimativa apresenta flutuações estatísticas e pode, inclusive, aproximar-se temporariamente do valor exato antes de se afastar novamente. Portanto, o programa determina o primeiro instante, para a sequência pseudoaleatória e a semente utilizadas, em que o erro absoluto se torna inferior a

$$10^{-4}.$$

De forma geral, métodos de Monte Carlo apresentam uma convergência estatística relativamente lenta, caracterizada por flutuações que diminuem tipicamente como

$$\frac{1}{\sqrt{N_{\text{total}}}}.$$

Ainda assim, o método ilustra de maneira direta a relação entre probabilidades, frequências relativas e áreas geométricas. Neste caso, a estimativa de π surge naturalmente da proporção de pontos aleatórios que pertencem ao círculo unitário em relação ao número total de pontos distribuídos no quadrado.

1.4 (d)

1.4.1 Descrição do Problema

O objetivo é utilizar o método de Monte Carlo para calcular numericamente integrais definidas.

O método é baseado no Teorema do Valor Médio para integrais:

$$\int_a^b f(x) dx = (b - a) \langle f \rangle.$$

A média da função é estimada por meio de N pontos aleatórios

$$x_i \in [a, b],$$

de forma que

$$\langle f \rangle \approx \frac{1}{N} \sum_{i=1}^N f(x_i).$$

Assim, a estimativa numérica da integral é dada por

$$\int_a^b f(x) dx \approx \frac{b - a}{N} \sum_{i=1}^N f(x_i).$$

Devem ser calculadas as integrais

$$\int_0^1 \frac{4}{x^2 + 1} dx$$

e

$$\int_1^{10} \frac{1}{x} dx.$$

Os resultados devem ser obtidos com três casas decimais de precisão, determinando-se também o valor de N necessário.

1.4.2 Estratégia de Solução

A implementação utiliza o método de Monte Carlo para estimar as duas integrais a partir da média dos valores das respectivas funções avaliadas em pontos aleatórios uniformemente distribuídos nos intervalos de integração.

Inicialmente, foi definida uma semente fixa,

$$\text{iseed} = 42,$$

de modo a garantir a reprodutibilidade dos resultados. A função ‘numero_aleatorio_entre()’ realiza uma transformação linear dos valores produzidos por ‘rand()’, permitindo gerar pontos uniformemente distribuídos em um intervalo arbitrário,

$$[a, b].$$

Para a primeira integral, foram utilizados os limites

$$a_i = 0, \quad b_i = 1,$$

e a função

$$f_i(x) = \frac{4}{x^2 + 1}.$$

A cada iteração, um novo ponto aleatório é gerado no intervalo de integração e o valor da função nesse ponto é adicionado ao somatório acumulado. A estimativa da integral é então atualizada segundo

$$I_i(N) = (b_i - a_i) \frac{1}{N} \sum_{j=1}^N f_i(x_j).$$

O valor exato utilizado como referência foi

$$I_{i,\text{real}} = \pi \approx 3.14159.$$

Para a segunda integral, o mesmo procedimento foi aplicado ao intervalo

$$a_{ii} = 1, \quad b_{ii} = 10,$$

com a função

$$f_{ii}(x) = \frac{1}{x}.$$

Nesse caso, o valor exato é

$$I_{ii,\text{real}} = \ln(10) \approx 2.30259.$$

Em ambos os casos, a estimativa é atualizada a cada novo ponto gerado. O programa é interrompido para cada integral quando o erro absoluto em relação ao valor de referência se torna inferior à tolerância estabelecida,

$$\epsilon = 10^{-3}.$$

Assim, o critério utilizado para aceitar o resultado é

$$|I_{\text{MC}} - I_{\text{real}}| < 10^{-3}.$$

O número de pontos correspondente à primeira ocorrência dessa condição é registrado como o valor de N necessário para atingir a precisão especificada para a sequência pseudoaleatória utilizada.

```

1      program tarefald
2
3
4      integer*4 N
5      real*8 x, somatorio_fx, epsilon, integral_i, integral_ii
6      parameter(epsilon = (10d0)**(-3)) ! precisão
7
8      real*8 numero_aleatorio_entre, funcao_i, funcao_ii ! funções
9      real*8 a_i, b_i, a_ii, b_ii ! Intervalos de integração
10     real*8 integral_i_real, integral_ii_real ! Valores reais das integrais
11
12     integer*4 iseed
13     parameter(iseed = 42)
14     call srand(iseed)
15
16     ! Integral (i)
17     print *, 'Resultados para a integral (i):'
18     a_i = 0d0
19     b_i = 1d0
20     integral_i_real = 3.14159265358979323 ! pi
21     somatorio_fx = 0
22     do N = 1, 10**8
23         x = numero_aleatorio_entre(a_i, b_i)
24         somatorio_fx = somatorio_fx + funcao_i(x) ! Para calcular a média
25         integral_i = (b_i - a_i) * somatorio_fx / N ! Teorema do Valor Médio
26         ! Quando o erro é menor que epsilon, mostra os resultados finais
27         if (abs(integral_i - integral_i_real) .LT. epsilon) then
28             print *, 'O numero de pontos necessario eh', N
29             print *, 'A integral estimada eh', integral_i
30             goto 10
31         end if
32     end do
33
34 10    continue
35
36     ! Integral (ii)
37     print *, 'Resultados para a integral (ii):'
38     a_ii = 1d0
39     b_ii = 10d0
40     integral_ii_real = log(10d0) ! ln(10)
41     somatorio_fx = 0
42     do N = 1, 10**8
43         x = numero_aleatorio_entre(a_ii, b_ii)
44         somatorio_fx = somatorio_fx + funcao_ii(x)
45         integral_ii = (b_ii - a_ii) * somatorio_fx / N
46         if (abs(integral_ii - integral_ii_real) .LT. epsilon) then
47             print *, 'O numero de pontos necessario eh', N
48             print *, 'A integral estimada eh', integral_ii
49             goto 20
50         end if
51     end do

```

```

52
53 20    continue
54
55    end
56
57    function numero_aleatorio_entre(inicio, fim)
58        ! Gera um numero aleatorio uniformemente distribuido entre inicio e fim.
59        ! Entrada: inicio e fim - limites inferior e superior do intervalo.
60        ! Saida: numero_aleatorio_entre - numero aleatorio gerado no intervalo.
61        real*8 inicio, fim, numero_aleatorio_entre
62
63        numero_aleatorio_entre = (fim - inicio) * rand() + inicio ! bijecao linear
        entre o intervalo [0,1) e o intervalo [inicio, fim]
64
65        return
66    end
67
68    function funcao_i(x)
69        ! f(x) = 4 / (x^2 + 1)
70        real*8 x, funcao_i
71        funcao_i = 4d0 / (x**2 + 1d0)
72        return
73    end
74
75    function funcao_ii(x)
76        ! f(x) = 1 / x
77        real*8 x, funcao_ii
78        funcao_ii = 1d0 / x
79        return
80    end

```

1.4.3 Resultados Obtidos

Para a primeira integral, o método de Monte Carlo produziu o resultado

$$N = 663$$

e

$$I_i = 3.1423260814918157.$$

O valor exato da integral é

$$\int_0^1 \frac{4}{x^2 + 1} dx = 4 \arctan(x) \Big|_0^1 = \pi \approx 3.141592653589793.$$

Portanto, o erro absoluto da estimativa obtida é aproximadamente

$$|I_i - \pi| \approx 7.33 \times 10^{-4},$$

valor inferior à tolerância estabelecida,

$$\epsilon = 10^{-3}.$$

Assim, a precisão solicitada foi atingida após a geração de 663 pontos aleatórios. Para a segunda integral, foi obtido

$$N = 250675$$

e

$$I_{ii} = 2.3035829519631865.$$

O resultado analítico correspondente é

$$\int_1^{10} \frac{1}{x} dx = \ln(x) \Big|_1^{10} = \ln(10) \approx 2.302585092994046.$$

O erro absoluto obtido é aproximadamente

$$|I_{ii} - \ln(10)| \approx 1.00 \times 10^{-3}.$$

Considerando o valor de referência utilizado no programa,

$$\ln(10) \approx 2.30259,$$

o resultado satisfaz o critério de parada implementado, uma vez que o erro em relação a esse valor truncado é inferior a

$$10^{-3}.$$

Observa-se uma diferença significativa entre os números de pontos necessários para as duas integrais.

Essa diferença decorre das flutuações estatísticas inerentes ao método de Monte Carlo e da variância dos valores da função amostrada em cada intervalo. A função

$$f_i(x) = \frac{4}{1+x^2}$$

é limitada entre 2 e 4 no intervalo de integração, enquanto

$$f_{ii}(x) = \frac{1}{x}$$

apresenta uma variação relativa significativa ao longo do intervalo $[1, 10]$. Como a precisão da estimativa de Monte Carlo está diretamente relacionada à dispersão dos valores amostrados, diferentes funções podem exigir números bastante distintos de pontos para atingir uma mesma tolerância.

Saída:

Resultados para a integral (i):

O numero de pontos necessario eh 663

A integral estimada eh 3.1423260814918157

Resultados para a integral (ii):

O numero de pontos necessario eh 250675

A integral estimada eh 2.3035829519631865

2 Problema 2: Decaimento Radioativo

2.1 (a)

2.1.1 Descrição do Problema

O objetivo é simular o decaimento de um conjunto de

$$N_0$$

núcleos radioativos de vida média τ .

A probabilidade de decaimento em um intervalo infinitesimal de tempo é dada por

$$dP = \lambda dt,$$

onde

$$\lambda = \frac{1}{\tau}.$$

A simulação utiliza um intervalo de tempo pequeno

$$\delta t \ll \tau,$$

de modo que a probabilidade de um núcleo decair durante cada intervalo seja

$$p = \frac{\delta t}{\tau}.$$

Para cada núcleo ainda presente, gera-se um número aleatório r . O núcleo decai quando

$$r < p.$$

O processo deve ser repetido até um tempo máximo

$$t_{\max} \gg \tau.$$

Além disso, o programa deve ser capaz de repetir a simulação k vezes utilizando diferentes sequências de números aleatórios.

2.1.2 Estratégia de Solução

A simulação foi implementada utilizando o método de Monte Carlo para reproduzir o caráter probabilístico do decaimento radioativo. Foram considerados inicialmente

$$N_0 = 10^3$$

núcleos radioativos, com vida média

$$\tau = 1.$$

O intervalo total de simulação foi escolhido como

$$t_{\max} = 10,$$

e dividido em 1000 intervalos de tempo. Assim, o passo temporal utilizado é

$$\delta t = \frac{t_{\max}}{1000} = 0.01.$$

Em cada intervalo temporal, todos os núcleos ainda presentes são analisados individualmente. Para cada núcleo, a função ‘rand()’ gera um número pseudoaleatório uniformemente distribuído. Caso o valor sorteado seja menor que a probabilidade p , o núcleo é considerado decaído durante aquele intervalo.

A função ‘get_decaimentos()’ realiza esse procedimento e retorna o número total de núcleos que decaem durante um determinado intervalo δt . Se o número de núcleos presentes no início do intervalo for

$$N(t),$$

o número de núcleos restantes após o intervalo é calculado por

$$N(t + \delta t) = N(t) - N_{\text{decaimentos}}.$$

A sub-rotina ‘simulacao_decaimento()’ repete esse processo para todos os intervalos temporais, armazenando no vetor ‘Nt’ o número de núcleos restantes em cada instante.

O procedimento completo é repetido

$$k = 10^3$$

vezes. Cada realização utiliza a continuação da sequência de números pseudoaleatórios gerada a partir da mesma inicialização, produzindo, assim, diferentes trajetórias estocásticas do processo de decaimento. Os resultados de cada simulação são armazenados em uma linha do arquivo de saída, enquanto as colunas correspondem aos diferentes instantes de tempo.

A utilização de múltiplas realizações permite analisar posteriormente tanto o comportamento individual das trajetórias quanto as propriedades estatísticas do conjunto de simulações.

```
1
2      program tarefa2a
3
4      implicit none
5
6      character(50) arquivo_saida
7
8      integer*4 i, j
9      integer*4 k
10     integer*4 num_dts
11     integer*4 Nt(10**3)
12
13     integer*4 N0
14     real*8 tau, tmax, dt
15
16     integer*4 iseed
17     parameter(iseed = 42)
18     call srand(iseed)
19
```

```

20     arquivo_saida = 'tarefa-2a-saida-1.dat'
21     open(unit=10, file=arquivo_saida)
22
23     ! Define os parâmetros das simulações
24     NO = (10d0)**3
25     k = (10d0)**3
26     tau = 1d0
27     tmax = 10d0 ! tmax >> tau
28     dt = 0.01d0 ! dt << tmax
29
30     ! Calcula o número de intervalos de tempo da simulação
31     num_dts = int(tmax / dt)
32
33     ! Realiza k simulações independentes do decaimento radioativo
34     do i = 1, k
35         Nt(1) = NO
36         call simulacao_decaimento(Nt, tmax, dt, tau)
37         write(10, *) (Nt(j), j = 1, num_dts + 1) ! Linhas i são simulações,
colunas j são instantes de tempo
38
39     end do
40
41     close(10)
42
43     end
44
45     function get_decaimentos(N_atual, tau, dt)
46     ! Sorteia decaimentos radioativos de núcleos a partir
47     ! da probabilidade de decaimento dp = (1/tau) dt.
48     ! Entradas:
49     ! N_atual - número de núcleos
50     ! tau      - vida média
51     ! dt       - intervalo de tempo
52     ! Saída:
53     ! get_decaimentos - número de decaimentos no intervalo dt
54
55     integer*4 i, N_atual
56     integer*4 get_decaimentos
57     real*8 tau, dt, dp, num_aleatorio
58
59     dp = dt / tau
60     get_decaimentos = 0
61
62     ! Para cada núcleo intacto, sorteia se ele vai decair
63     do i = 1, N_atual
64         num_aleatorio = rand()
65         if (num_aleatorio .lt. dp) then
66             get_decaimentos = get_decaimentos + 1
67         endif
68     end do
69

```

```

70     return
71     end
72
73     subroutine simulacao_decaimento(Nt, tmax, dt, tau)
74         ! Simula o decaimento radioativo de N0 nucleos ao longo
75         ! do intervalo de tempo [0, tmax], dividindo-o em
76         ! intervalos de duracao dt. O numero de decaimentos é dado pela funcao
77         get_decaimentos.
78         ! Entradas:
79         ! Nt    - vetor que armazena o numero de nucleos em cada
80         !        instante de tempo. O primeiro elemento Nt(1)
81         !        corresponde ao instante t = 0.
82         ! tmax - tempo maximo da simulacao.
83         ! dt   - intervalo de tempo.
84         ! tau  - vida media.
85         ! Saida:
86         ! Nt    - vetor contendo o numero de nucleos restantes em
87         !        cada instante de tempo da simulacao. O elemento
88         !        Nt(indice_tempo + 1) é obtido a partir de
89         !        Nt(indice_tempo) após os decaimentos ocorridos
90         !        durante o intervalo dt.
91
92         integer*4 indice_tempo, num_dts
93         real*8 tau, tmax, dt
94         integer*4 N_atual
95         integer*4 Nt(10**3)
96         integer*4 decaimentos, get_decaimentos
97
98         num_dts = int(tmax / dt)
99
100        ! Loop para cada intervalo dt no intervalo [0, tmax]
101        do indice_tempo = 1, num_dts
102            N_atual = Nt(indice_tempo)
103            decaimentos = get_decaimentos(N_atual, tau, dt)
104            Nt(indice_tempo + 1) = N_atual - decaimentos ! Subtrai os núcleos
105            que decaíram no intervalo
106        end do
107
108    end

```

2.1.3 Resultados Obtidos

A simulação produz $k = 10^3$ trajetórias independentes para o número de núcleos radioativos restantes ao longo do tempo. Cada trajetória apresenta flutuações estatísticas, pois o instante de decaimento de cada núcleo é determinado individualmente por sorteios pseudoaleatórios.

Apesar desse caráter estocástico, espera-se que o comportamento médio do conjunto de simulações reproduza a lei exponencial do decaimento radioativo,

$$N(t) = N_0 e^{-t/\tau}.$$

Essa expressão pode ser obtida a partir da equação diferencial que descreve o decaimento,

$$\frac{dN}{dt} = -\lambda N,$$

com

$$\lambda = \frac{1}{\tau}.$$

Para os parâmetros utilizados na simulação, a previsão teórica é

$$N(t) = 10^3 e^{-t}.$$

Em particular, ao final do intervalo simulado,

$$t = t_{\max} = 10 = 10\tau,$$

o número médio esperado de núcleos restantes é

$$N(10) = N_0 e^{-10} \approx 0.0454.$$

Portanto, após um intervalo correspondente a dez vidas médias, espera-se que praticamente todos os núcleos tenham decaído. Embora cada uma das 10^3 simulações possa apresentar pequenas diferenças durante a evolução temporal, ao final do intervalo o número de núcleos sobreviventes deve ser, em geral, muito próximo de zero.

As diferenças entre as trajetórias individuais são consequência das flutuações estatísticas inerentes ao processo de decaimento. Para um número finito de núcleos, o número de decaimentos ocorridos em cada intervalo temporal não é exatamente igual ao seu valor médio esperado. No entanto, quando um grande número de realizações é considerado, essas flutuações tendem a ser suavizadas, permitindo recuperar o comportamento exponencial previsto teoricamente.

O passo temporal utilizado,

$$\delta t = 0.01,$$

é pequeno em comparação com a vida média,

$$\tau = 1,$$

resultando em

$$\frac{\delta t}{\tau} = 0.01.$$

Assim, a probabilidade de decaimento durante cada intervalo é suficientemente pequena para que o procedimento discreto constitua uma aproximação adequada ao processo contínuo de decaimento radioativo.

Além disso, a escolha

$$t_{\max} = 10\tau$$

permite acompanhar o processo durante um intervalo suficientemente longo para observar praticamente todo o decaimento da população inicial.

Dessa forma, a simulação ilustra como uma lei determinística para o valor médio de um sistema,

$$N(t) = N_0 e^{-t/\tau},$$

pode emergir a partir do comportamento probabilístico de um grande número de eventos individuais.

2.2 (b)

2.2.1 Descrição do Problema

Neste item, deve ser realizada a simulação para

$$N_0 = 10^3$$

núcleos e

$$k = 10^3$$

repetições.

Devem ser escolhidos valores apropriados para

$$t_{\max}$$

e

$$\delta t,$$

justificando-se essas escolhas.

O resultado deve apresentar as curvas do número de núcleos restantes

$$N(t)$$

para as dez primeiras repetições, juntamente com o valor médio

$$\langle N(t) \rangle$$

calculado a partir das k simulações.

As barras de erro da média devem ser estimadas por

$$\frac{\sigma(t)}{\sqrt{k}},$$

onde $\sigma(t)$ é o desvio padrão dos resultados no instante t .

2.2.2 Estratégia de Solução

Foram realizadas

$$k = 10^3$$

simulações independentes do decaimento de um conjunto inicial de

$$N_0 = 10^3$$

núcleos radioativos. A vida média utilizada foi

$$\tau = 10.$$

Para observar adequadamente o processo de decaimento, foi escolhido o tempo máximo

$$t_{\max} = 30 = 3\tau.$$

Essa escolha permite acompanhar o sistema durante três vidas médias. Nesse instante, o valor teórico esperado para o número de núcleos restantes é

$$N(3\tau) = N_0 e^{-3} \approx 49.8.$$

Portanto, ao final da simulação, aproximadamente 5% dos núcleos iniciais ainda permanecem, permitindo observar uma parcela significativa do processo de decaimento.

O intervalo temporal foi escolhido como

$$\delta t = 0.1.$$

Como

$$\frac{\delta t}{\tau} = \frac{0.1}{10} = 0.01,$$

a probabilidade de decaimento de um núcleo durante cada intervalo é pequena,

$$p = \frac{\delta t}{\tau} = 0.01.$$

Dessa forma, a discretização temporal constitui uma boa aproximação para o processo contínuo de decaimento e, simultaneamente, fornece resolução temporal suficiente para observar a evolução de

$$N(t).$$

Em cada uma das k simulações, o vetor ‘Nt’ armazena o número de núcleos restantes em todos os instantes de tempo. Para cada intervalo, a função ‘get_decaimentos()’ analisa individualmente os núcleos presentes e sorteia seus possíveis decaimentos com probabilidade

$$p = \frac{\delta t}{\tau}.$$

Após a realização das k simulações, foram acumuladas, para cada instante de tempo, as quantidades

$$\sum_{i=1}^k N_i(t)$$

e

$$\sum_{i=1}^k N_i^2(t).$$

A partir desses valores, o número médio de núcleos restantes foi calculado por

$$\langle N(t) \rangle = \frac{1}{k} \sum_{i=1}^k N_i(t).$$

O valor médio do quadrado do número de núcleos é dado por

$$\langle N^2(t) \rangle = \frac{1}{k} \sum_{i=1}^k N_i^2(t).$$

Assim, o desvio padrão entre as diferentes simulações foi obtido por

$$\sigma(t) = \sqrt{\langle N^2(t) \rangle - \langle N(t) \rangle^2}.$$

Finalmente, a incerteza associada à média foi estimada pelo erro padrão,

$$\Delta \langle N(t) \rangle = \frac{\sigma(t)}{\sqrt{k}}.$$

Além das quantidades estatísticas, o programa armazena explicitamente as dez primeiras realizações no vetor 'N10'. Dessa forma, é possível comparar no gráfico as trajetórias individuais com o comportamento médio do conjunto de simulações.

O arquivo de saída é organizado de modo que cada linha corresponda a um instante de tempo. As colunas contêm, respectivamente, o tempo t , os valores de $N(t)$ das dez primeiras simulações, o valor médio

$$\langle N(t) \rangle$$

e sua respectiva barra de erro.

```

1
2 program tarefa2b
3 ! Gera um arquivo de saida contendo, em cada linha,
4 ! o instante de tempo t, os valores de N(t) das dez
5 ! primeiras simulacoes, o valor medio N_medio(t) obtido
6 ! das k simulacoes e a barra de erro correspondente.
7 ! Formato: t, N1, N2, ..., N10, N_medio, erro
8
9 implicit none
10
11 character*50 arquivo_saida
12
13 integer*4 i, j
14 integer*4 k
15 integer*4 num_dts
16 integer*4 Nt(10**3)
17 integer*4 N10(10,10**3)
18 integer*4 soma_N(10**3)
19
20 integer*4 N0
21 real*8 soma_N2(10**3)
22 real*8 N_medio
23 real*8 N_quadratico_medio
24 real*8 desvio_padrao

```

```

25  real*8 erro
26  real*8 tau, tmax, dt, t
27
28  integer*4 iseed
29  parameter(iseed = 42)
30
31  call srand(iseed)
32
33  arquivo_saida = 'tarefa-2b-saida-1.dat'
34  open(unit=10, file=arquivo_saida)
35
36  N0 = (10d0)**3
37  k = (10d0)**3
38
39  tau = 10d0
40  tmax = 30d0
41  dt = 0.1d0
42
43  num_dts = int(tmax / dt)
44
45  do j = 1, num_dts + 1
46      soma_N(j) = 0
47      soma_N2(j) = 0d0
48  end do
49
50  ! Preenche as 10 primeiras colunas com as 10 primeiras simulacoes
51  do i = 1, k
52
53      Nt(1) = N0
54
55      call simulacao_decaimento(Nt, tmax, dt, tau)
56
57      do j = 1, num_dts + 1
58
59          soma_N(j) = soma_N(j) + Nt(j)
60
61          soma_N2(j) = soma_N2(j) +
62  &              dble(Nt(j))**2
63
64          if (i .le. 10) then
65              N10(i,j) = Nt(j)
66          endif
67
68      end do
69
70  end do
71
72  ! Preenche as ultimas colunas com N_medio e o erro
73  do j = 1, num_dts + 1
74
75      t = (j - 1) * dt

```

```

76
77     N_medio = dble(soma_N(j)) / dble(k)
78
79     N_quadratico_medio = soma_N2(j) / dble(k)
80
81     desvio_padrao = dsqrt(N_quadratico_medio -
82 &         N_medio**2)
83
84     erro = desvio_padrao / dsqrt(dble(k))
85
86     write(10,*) t,
87 &         (N10(i,j), i = 1,10),
88 &         N_medio, erro
89
90 end do
91
92 close(10)
93
94 end
95
96 function get_decaimentos(N_atual, tau, dt)
97 ! Sorteia decaimentos radioativos de nucleos a partir
98 ! da probabilidade de decaimento dp = (1/tau) dt.
99 ! Entradas:
100 ! N_atual - numero de nucleos
101 ! tau      - vida media
102 ! dt       - intervalo de tempo
103 ! Saida:
104 ! get_decaimentos - numero de decaimentos no intervalo dt
105
106 integer*4 i, N_atual
107 integer*4 get_decaimentos
108 real*8 tau, dt, dp, num_aleatorio
109
110 dp = dt / tau
111 get_decaimentos = 0
112
113 do i = 1, N_atual
114     num_aleatorio = rand()
115     if (num_aleatorio .lt. dp) then
116         get_decaimentos = get_decaimentos + 1
117     endif
118 end do
119
120 return
121 end
122
123 subroutine simulacao_decaimento(Nt, tmax, dt, tau)
124 ! Simula o decaimento radioativo de N0 nucleos ao longo
125 ! do intervalo de tempo [0, tmax], dividindo-o em
126 ! intervalos de duracao dt. 0 numero de decaimentos e

```

```

127 ! dado pela funcao get_decaimentos.
128 ! Entradas:
129 ! Nt - vetor que armazena o numero de nucleos em cada
130 !     instante de tempo. O primeiro elemento Nt(1)
131 !     corresponde ao instante t = 0.
132 ! tmax - tempo maximo da simulacao.
133 ! dt - intervalo de tempo.
134 ! tau - vida media.
135 ! Saida:
136 ! Nt - vetor contendo o numero de nucleos restantes em
137 !     cada instante de tempo da simulacao. O elemento
138 !     Nt(indice_tempo + 1) e obtido a partir de
139 !     Nt(indice_tempo) apos os decaimentos ocorridos
140 !     durante o intervalo dt.
141
142 integer*4 indice_tempo, num_dts
143 real*8 tau, tmax, dt
144 integer*4 N_atual
145 integer*4 Nt(10**3)
146 integer*4 decaimentos, get_decaimentos
147
148 num_dts = int(tmax / dt)
149
150 ! Loop para cada intervalo dt no intervalo [0, tmax]
151 do indice_tempo = 1, num_dts
152     N_atual = Nt(indice_tempo)
153     decaimentos = get_decaimentos(N_atual, tau, dt)
154     Nt(indice_tempo + 1) =
155 &         N_atual - decaimentos
156 end do
157
158 end

```

2.2.3 Resultados Obtidos

A Figura 5 apresenta a evolução temporal do número de núcleos radioativos restantes para as dez primeiras simulações, juntamente com a média obtida a partir de

$$k = 10^3$$

realizações independentes.

As curvas individuais apresentam flutuações estatísticas, uma vez que o processo de decaimento é intrinsecamente probabilístico. Em cada realização, os núcleos decaem em instantes diferentes, produzindo trajetórias distintas para

$$N(t).$$

Entretanto, todas as trajetórias apresentam o comportamento geral esperado de diminuição do número de núcleos ao longo do tempo.

Ao considerar a média sobre as 10^3 simulações, as flutuações individuais são significativamente reduzidas. O comportamento médio deve aproximar a lei exponencial teórica

do decaimento radioativo,

$$\langle N(t) \rangle = N_0 e^{-t/\tau}.$$

Com os parâmetros utilizados,

$$N_0 = 10^3$$

e

$$\tau = 10,$$

a previsão teórica torna-se

$$\langle N(t) \rangle = 10^3 e^{-t/10}.$$

A boa concordância esperada entre a curva média e o comportamento exponencial evidencia que a lei macroscópica do decaimento radioativo emerge da média de um grande número de eventos individuais aleatórios.

As barras de erro representam a incerteza estatística associada à estimativa da média,

$$\Delta \langle N(t) \rangle = \frac{\sigma(t)}{\sqrt{k}}.$$

Como foram utilizadas

$$k = 10^3$$

simulações, o erro da média é consideravelmente menor que a dispersão observada entre as trajetórias individuais. Isso explica por que a curva média apresenta um comportamento muito mais suave e bem definido do que cada realização isolada.

Também se observa que a dispersão absoluta entre as simulações tende a diminuir à medida que o número de núcleos restantes se torna menor. No entanto, como o número de núcleos também diminui, as flutuações relativas podem tornar-se mais relevantes nos tempos finais da simulação.

A escolha de

$$t_{\max} = 30 = 3\tau$$

permite observar claramente o comportamento exponencial durante uma parcela significativa do decaimento. Ao final da simulação, o valor médio teoricamente esperado é

$$\langle N(30) \rangle = 10^3 e^{-3} \approx 49.8.$$

Portanto, a simulação acompanha o sistema desde o conjunto inicial de 1000 núcleos até uma situação em que apenas uma pequena fração dos núcleos permanece.

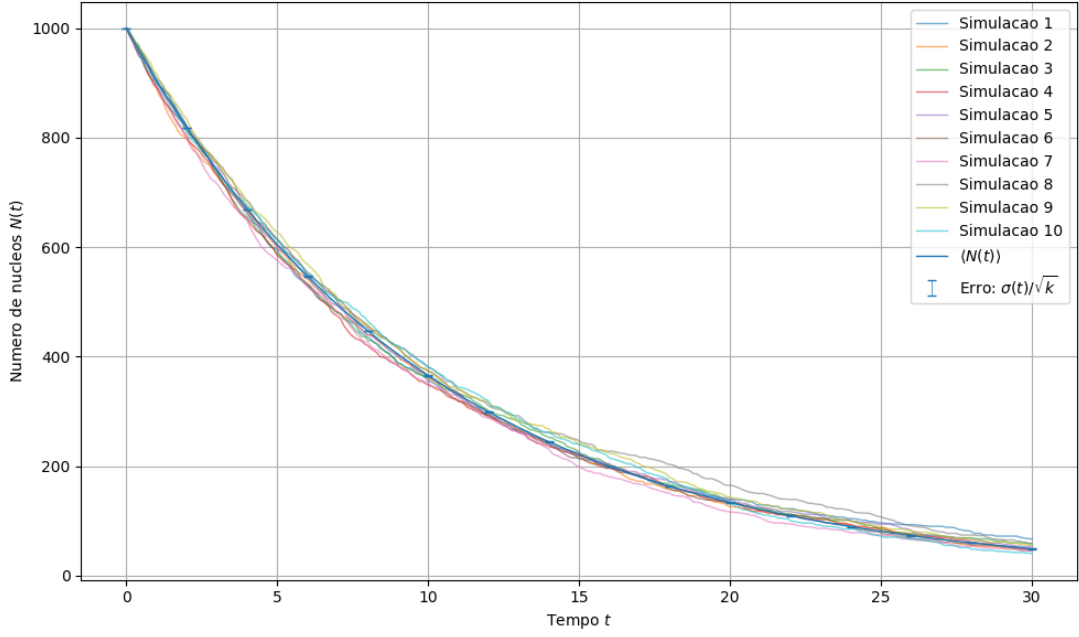


Figura 5: Número de núcleos radioativos restantes em função do tempo $N(t)$ para as dez primeiras simulações e para o valor médio obtido $\langle N(t) \rangle$ a partir de k repetições, com barras de erro dadas por $\frac{\sigma(t)}{\sqrt{k}}$.

2.3 (c)

2.3.1 Descrição do Problema

O processo de decaimento radioativo pode ser descrito por uma distribuição exponencial

$$p(x) = \lambda e^{-\lambda x}, \quad x \geq 0.$$

Inicialmente, deve-se mostrar que

$$\lambda = \frac{1}{\langle x \rangle}.$$

Em seguida, o objetivo é gerar números aleatórios distribuídos segundo essa distribuição a partir de números aleatórios uniformes.

Para isso, utiliza-se a transformação

$$x = -\frac{1}{\lambda} \ln(1 - r),$$

onde

$$0 < r < 1$$

é uma variável aleatória uniforme.

Por fim, deve-se construir um histograma normalizado dos valores de x e compará-lo com a distribuição exponencial esperada.

2.3.2 Estratégia de Solução

Inicialmente, pode-se determinar a relação entre o parâmetro λ e o valor médio da distribuição exponencial. O valor esperado de x é definido por

$$\langle x \rangle = \int_0^{\infty} x p(x) dx.$$

Substituindo a expressão da distribuição exponencial,

$$p(x) = \lambda e^{-\lambda x},$$

obtem-se

$$\langle x \rangle = \int_0^{\infty} x \lambda e^{-\lambda x} dx = \lambda \int_0^{\infty} x e^{-\lambda x} dx.$$

Para resolver essa integral, utiliza-se integração por partes. Escolhendo

$$u = x, \quad dv = e^{-\lambda x} dx,$$

tem-se

$$du = dx$$

e

$$v = \int e^{-\lambda x} dx = -\frac{1}{\lambda} e^{-\lambda x}.$$

Aplicando a fórmula

$$\int u dv = uv - \int v du,$$

segue que

$$\int_0^{\infty} x e^{-\lambda x} dx = \left[-\frac{x}{\lambda} e^{-\lambda x} \right]_0^{\infty} + \frac{1}{\lambda} \int_0^{\infty} e^{-\lambda x} dx.$$

O primeiro termo é nulo nos dois limites: para $x = 0$, ele é evidentemente zero, e, quando $x \rightarrow \infty$, o fator exponencial $e^{-\lambda x}$ decai mais rapidamente do que x cresce. Assim,

$$\left[-\frac{x}{\lambda} e^{-\lambda x} \right]_0^{\infty} = 0.$$

Resta, portanto,

$$\int_0^{\infty} x e^{-\lambda x} dx = \frac{1}{\lambda} \int_0^{\infty} e^{-\lambda x} dx.$$

Calculando a integral restante,

$$\int_0^{\infty} e^{-\lambda x} dx = \left[-\frac{1}{\lambda} e^{-\lambda x} \right]_0^{\infty} = \frac{1}{\lambda}.$$

Logo,

$$\int_0^{\infty} x e^{-\lambda x} dx = \frac{1}{\lambda^2}.$$

Finalmente, lembrando o fator λ colocado inicialmente em evidência,

$$\langle x \rangle = \lambda \left(\frac{1}{\lambda^2} \right) = \frac{1}{\lambda}.$$

Portanto,

$$\lambda = \frac{1}{\langle x \rangle}.$$

Para gerar números aleatórios com essa distribuição a partir de uma variável uniforme, foi utilizado o método da transformação inversa. A função de distribuição acumulada da exponencial é

$$F(x) = \int_0^x \lambda e^{-\lambda u} du.$$

Calculando a integral,

$$F(x) = 1 - e^{-\lambda x}.$$

Se r é uma variável aleatória uniformemente distribuída no intervalo

$$0 < r < 1,$$

então, pelo método da transformação inversa, identifica-se

$$r = F(x) = 1 - e^{-\lambda x}.$$

Reorganizando essa expressão,

$$e^{-\lambda x} = 1 - r,$$

e, aplicando o logaritmo natural,

$$-\lambda x = \ln(1 - r).$$

Assim, obtém-se

$$x = -\frac{1}{\lambda} \ln(1 - r),$$

que é a expressão utilizada para gerar os números aleatórios exponencialmente distribuídos.

Na implementação, foi escolhido o parâmetro

$$\lambda = 0.1,$$

de modo que o valor médio teórico da distribuição seja

$$\langle x \rangle = \frac{1}{0.1} = 10.$$

Foram gerados

$$N = 10^4$$

valores independentes. Para cada valor, a função ‘rand()’ fornece um número pseudo-aleatório uniforme r , que é então transformado pela função ‘gerar_x()’ segundo a relação inversa da distribuição acumulada.

A semente do gerador foi fixada como

`iseed = 42,`

permitindo a reprodutibilidade dos resultados. Os valores gerados foram armazenados em um arquivo de saída e posteriormente utilizados para construir um histograma normalizado. A normalização permite comparar diretamente a densidade observada dos dados com a distribuição teórica

$$p(x) = \lambda e^{-\lambda x}.$$

```
1
2  program tarefa2c
3
4  character(50) arquivo_saida
5
6  real*8 lambda
7  real*8 gerar_x, x
8
9  integer*4 num_dados
10
11 integer*4 iseed
12 parameter(iseed = 42)
13
14 call srand(iseed)
15
16 arquivo_saida = 'tarefa-2c-saida-1.dat'
17 open(unit=10, file=arquivo_saida)
18
19 lambda = 0.1d0
20 num_dados = 10**4
21
22 do i = 1, num_dados
23     x = gerar_x(lambda)
24     write(10, *) x
25 end do
26
27 close(10)
28
29 end
30
31 function gerar_x(lambda)
32 ! Gera um número aleatório seguindo uma distribuição exponencial de parâmetro
   lambda.
33 ! Entrada: lambda, parâmetro positivo da distribuição exponencial.
34 ! Saída: gerar_x, valor aleatório gerado segundo  $p(x) = \lambda e^{-\lambda x}$ .
35 real*8 lambda
36 real*8 gerar_x
```

```

37  r = rand()
38  gerar_x = - (1d0/lambda) * log(1d0-r)
39  return
40  end

```

2.3.3 Resultados Obtidos

A Figura 6 apresenta o histograma normalizado dos

$$N = 10^4$$

valores gerados pela transformação inversa, juntamente com a distribuição exponencial teórica.

Como esperado, observa-se uma concentração maior de valores próximos de

$$x = 0,$$

seguida por uma diminuição aproximadamente exponencial da frequência à medida que x aumenta. Esse comportamento é característico da distribuição

$$p(x) = \lambda e^{-\lambda x}.$$

Para o parâmetro utilizado,

$$\lambda = 0.1,$$

a função teórica comparada ao histograma é

$$p(x) = 0.1e^{-0.1x}.$$

O valor máximo da densidade ocorre em

$$x = 0,$$

onde

$$p(0) = \lambda = 0.1.$$

À medida que x aumenta, a probabilidade de observar valores naquela região diminui exponencialmente. Entretanto, a distribuição possui suporte infinito, de modo que, embora valores muito grandes sejam progressivamente menos prováveis, eles continuam sendo possíveis.

As pequenas diferenças entre as barras do histograma e a curva teórica são consequências das flutuações estatísticas associadas à geração de um número finito de dados. Como foram utilizados

$$10^4$$

valores, espera-se uma boa concordância global entre o histograma e a distribuição exponencial, embora flutuações locais ainda estejam presentes.

O resultado confirma que a transformação

$$x = -\frac{1}{\lambda} \ln(1 - r)$$

permite converter corretamente números aleatórios uniformemente distribuídos em números aleatórios com distribuição exponencial. Esse procedimento é particularmente importante no contexto do decaimento radioativo, pois a distribuição exponencial descreve a distribuição dos tempos de espera até o decaimento de um núcleo individual.

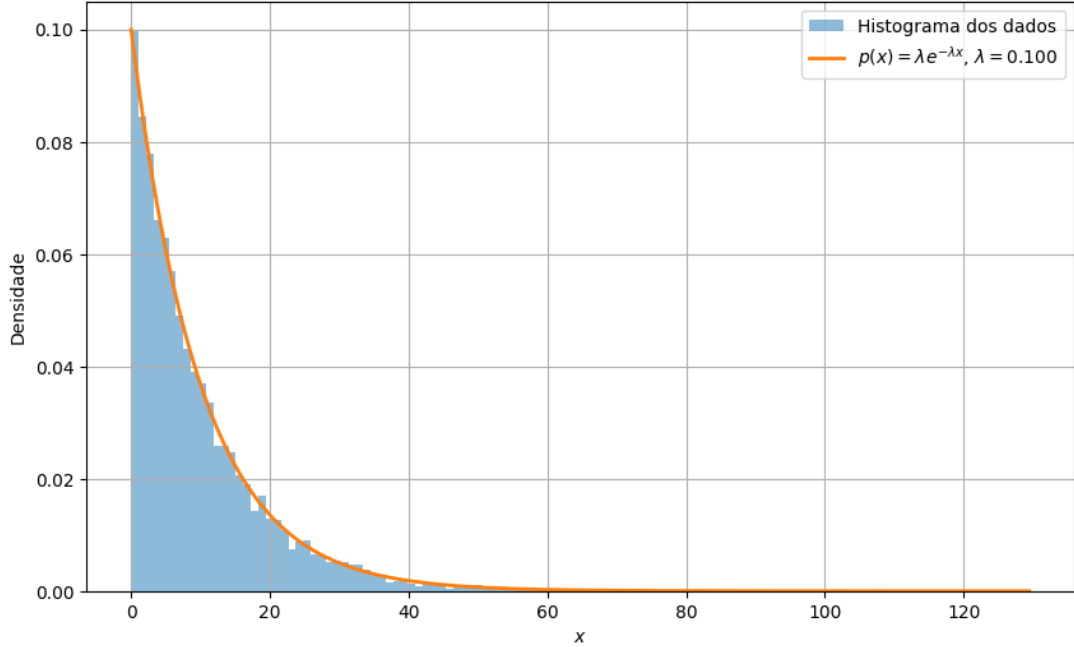


Figura 6: Histograma normalizado da variável x , comparado com a distribuição exponencial esperada.

3 Problema 3: Passeio Aleatório

3.1 (a)

3.1.1 Descrição do Problema

O objetivo é investigar um passeio aleatório em uma dimensão.

A partícula parte inicialmente da origem,

$$x = 0,$$

e, a cada instante de tempo, realiza um deslocamento aleatório

$$\Delta x$$

no intervalo

$$-1 \leq \Delta x \leq 1.$$

Assim, a posição da partícula evolui segundo

$$x(t+1) = x(t) + \Delta x.$$

Devem ser calculadas as distribuições de posição

$$P(x)$$

para

$$t = 50, \quad t = 100, \quad t = 200.$$

Também devem ser calculados o deslocamento médio

$$\langle x \rangle$$

e o deslocamento quadrático médio

$$\langle x^2 \rangle$$

como funções do tempo para

$$t \leq 200.$$

A simulação deve ser repetida para k diferentes sequências de números aleatórios. Deve-se determinar o valor de k a partir do qual as estimativas das médias apresentam um erro da ordem de 10%.

3.1.2 Estratégia de Solução

A simulação foi realizada gerando diversos caminhos independentes para uma partícula inicialmente localizada na origem. Em cada passo temporal, a posição da partícula é atualizada pela relação

$$x(t+1) = x(t) + \Delta x,$$

em que o deslocamento Δx é uma variável aleatória uniformemente distribuída no intervalo

$$-1 \leq \Delta x \leq 1.$$

A função ‘delta()’ realiza a geração desse deslocamento a partir da função ‘rand()’, que produz números uniformemente distribuídos no intervalo $[0, 1)$. A transformação linear utilizada é

$$\Delta x = 2r - 1,$$

onde

$$0 \leq r \leq 1.$$

Assim, Δx passa a estar uniformemente distribuído no intervalo desejado. Foram gerados inicialmente até

$$k_{\text{total}} = 10^6$$

caminhos possíveis, cada um contendo

$$200$$

passos temporais. Para cada caminho, foram acumuladas, em cada instante t , as quantidades

$$\sum_{i=1}^k x_i(t),$$

$$\sum_{i=1}^k x_i^2(t)$$

e

$$\sum_{i=1}^k x_i^4(t).$$

Esses acumuladores permitem calcular as estimativas da posição média,

$$\langle x(t) \rangle = \frac{1}{k} \sum_{i=1}^k x_i(t),$$

do deslocamento quadrático médio,

$$\langle x^2(t) \rangle = \frac{1}{k} \sum_{i=1}^k x_i^2(t),$$

e também dos desvios padrão necessários para estimar os erros estatísticos. Para a variável x , o desvio padrão foi calculado como

$$\sigma_x(t) = \sqrt{\langle x^2(t) \rangle - \langle x(t) \rangle^2}.$$

De maneira análoga, o desvio padrão da variável x^2 foi obtido a partir de

$$\sigma_{x^2}(t) = \sqrt{\langle x^4(t) \rangle - \langle x^2(t) \rangle^2}.$$

Os erros estatísticos associados às médias foram então estimados pelo erro padrão,

$$\Delta \langle x(t) \rangle = \frac{\sigma_x(t)}{\sqrt{k}},$$

e

$$\Delta \langle x^2(t) \rangle = \frac{\sigma_{x^2}(t)}{\sqrt{k}}.$$

Foi estabelecida uma tolerância

$$\epsilon = 10^{-1}.$$

Após a geração de cada novo caminho, o programa verifica se os erros das estimativas de

$$\langle x(t) \rangle$$

e

$$\langle x^2(t) \rangle$$

são menores que ϵ para todos os instantes entre 1 e 200. O menor número de realizações que satisfaz simultaneamente esse critério é armazenado como

$$k_{\min}.$$

Além das médias temporais, as posições das partículas nos instantes

$$t = 50, \quad t = 100, \quad t = 200$$

são armazenadas para a construção das distribuições de posição.

Para um passeio aleatório simétrico, o valor médio teórico do deslocamento é

$$\langle x(t) \rangle = 0,$$

pois não existe direção preferencial para o movimento. Como os incrementos são independentes e possuem média nula,

$$\langle \Delta x \rangle = 0.$$

Por outro lado, para uma variável uniformemente distribuída entre -1 e 1 ,

$$\langle (\Delta x)^2 \rangle = \frac{1}{3}.$$

Portanto, para t passos independentes, espera-se que o deslocamento quadrático médio apresente comportamento linear,

$$\langle x^2(t) \rangle = t \langle (\Delta x)^2 \rangle = \frac{t}{3}.$$

Essa relação será utilizada para interpretar a evolução temporal observada numericamente.

```

1
2  program tarefa3a
3
4  integer*4 k, k_total, t, k_min
5  integer*4 precisao_ok
6
7  real*8 delta
8  real*8 x
9
10 real*8 soma_x(200)
11 real*8 soma_x2(200)
12 real*8 soma_x4(200)
13
14 real*8 pos50(10**6)
15 real*8 pos100(10**6)
16 real*8 pos200(10**6)
17
18 real*8 media_x
19 real*8 media_x2

```

```

20  real*8 media_x4
21
22  real*8 desvio_x
23  real*8 desvio_x2
24
25  real*8 erro_x
26  real*8 erro_x2
27
28  real*8 epsilon
29  parameter(epsilon = (10d0)**(-1))
30
31  ! Inicializa os acumuladores
32  do t = 1, 200
33      soma_x(t) = 0d0
34      soma_x2(t) = 0d0
35      soma_x4(t) = 0d0
36  end do
37
38  k_total = 10**6
39  k_min = 0
40
41  ! Gera k caminhos
42  do k = 1, k_total
43
44      x = 0d0
45
46      ! Cada caminho com 200 passos
47      do t = 1, 200
48
49          x = x + delta()
50
51          soma_x(t) = soma_x(t) + x
52          soma_x2(t) = soma_x2(t) + x**2
53          soma_x4(t) = soma_x4(t) + x**4
54
55          ! Distribuicoes P(x) para t = 50, 100, 200
56          if (t .eq. 50) then
57              pos50(k) = x
58          else if (t .eq. 100) then
59              pos100(k) = x
60          else if (t .eq. 200) then
61              pos200(k) = x
62          end if
63
64      end do
65
66      ! A cada caminho gerado, verifica a precisao para todos os tempos
67      if (k .ge. 2) then
68
69          precisao_ok = 1
70

```

```

71     do t = 1, 200
72
73         media_x = soma_x(t)/dble(k)
74         media_x2 = soma_x2(t)/dble(k)
75         media_x4 = soma_x4(t)/dble(k)
76
77         ! Desvio padrao de x
78         desvio_x = dsqrt(media_x2 - media_x**2)
79
80         ! Desvio padrao de x^2
81         desvio_x2 = dsqrt(media_x4 - media_x2**2)
82
83         ! Erro das medias
84         erro_x = desvio_x/dsqrt(dble(k))
85         erro_x2 = desvio_x2/dsqrt(dble(k))
86
87         ! Se qualquer tempo nao atingir a precisao,
88         ! este k ainda nao e suficiente
89         if (erro_x .gt. epsilon) then
90             precisao_ok = 0
91         end if
92
93         if (erro_x2 .gt. epsilon) then
94             precisao_ok = 0
95         end if
96
97     end do
98
99     ! Se todos os tempos atingiram a precisao,
100    ! encontramos o k necessario
101    if (precisao_ok .eq. 1) then
102        k_min = k
103        goto 100
104    end if
105
106    end if
107
108    end do
109
110    100    continue
111
112    print *, 'k necessario = ', k_min
113    print *, 'precisao = ', epsilon
114
115    ! Abrir arquivos
116    open(unit=10, file='tarefa-3a-saida-1.dat')
117    open(unit=20, file='tarefa-3a-saida-2.dat')
118    open(unit=30, file='tarefa-3a-saida-3.dat')
119
120    ! Histogramas
121    do k = 1, k_min

```

```

122     write(10, *) 50, pos50(k)
123 end do
124
125 do k = 1, k_min
126     write(10, *) 100, pos100(k)
127 end do
128
129 do k = 1, k_min
130     write(10, *) 200, pos200(k)
131 end do
132
133 ! Médias e médias quadráticas
134 do t = 1, 200
135     write(20, *) t, soma_x(t)/dble(k_min)
136     write(30, *) t, soma_x2(t)/dble(k_min)
137 end do
138
139 close(10)
140 close(20)
141 close(30)
142
143 end
144
145 function delta()
146 ! Gera um numero aleatorio uniformemente distribuido entre -1 e 1.
147 ! Entrada: Nenhuma.
148 ! Saida: delta - numero aleatorio gerado no intervalo.
149
150 real*8 delta
151
152 delta = 2d0 * rand() - 1d0 ! bijeção linear entre [0,1) e [-1,1]
153
154 return
155 end

```

3.1.3 Resultados Obtidos

O critério de precisão estabelecido no programa foi atingido para

$$k_{\min} = 885553.$$

Portanto, foram necessárias aproximadamente

$$8.86 \times 10^5$$

realizações independentes para que os erros estatísticos absolutos das estimativas de

$$\langle x(t) \rangle$$

e

$$\langle x^2(t) \rangle$$

fossem inferiores a

$$\epsilon = 0.1$$

para todos os instantes considerados.

A Figura 7 apresenta as distribuições de posição para

$$t = 50, \quad t = 100, \quad t = 200.$$

Como o deslocamento elementar não possui direção preferencial, espera-se que as distribuições sejam aproximadamente simétricas em torno da origem. Além disso, à medida que o número de passos aumenta, a dispersão das posições cresce.

Esse comportamento pode ser compreendido a partir do Teorema Central do Limite. Após um grande número de passos independentes, a posição

$$x(t) = \sum_{i=1}^t \Delta x_i$$

é formada pela soma de diversas variáveis aleatórias independentes. Assim, a distribuição de posições tende progressivamente a uma forma aproximadamente Gaussiana.

Como

$$\langle \Delta x \rangle = 0,$$

a distribuição permanece centrada aproximadamente em

$$x = 0.$$

A variância, entretanto, aumenta com o número de passos,

$$\sigma_x^2(t) = \langle x^2(t) \rangle - \langle x(t) \rangle^2 \approx \frac{t}{3}.$$

Consequentemente, espera-se que os histogramas se tornem mais largos à medida que o tempo aumenta. Esse comportamento é observado ao comparar as distribuições correspondentes a $t = 50$, $t = 100$ e $t = 200$.

A Figura 8 mostra a evolução do deslocamento quadrático médio. Para o passeio considerado, a previsão teórica é

$$\langle x^2(t) \rangle = \frac{t}{3}.$$

Assim, espera-se uma dependência linear entre o deslocamento quadrático médio e o tempo. O comportamento observado no gráfico está de acordo com essa previsão.

De maneira geral, pode-se escrever esse comportamento na forma

$$\langle x^2(t) \rangle \propto t.$$

Isso mostra que o deslocamento típico da partícula cresce como

$$|x|_{\text{típico}} \sim \sqrt{t},$$

e não linearmente com o tempo. Essa é uma característica fundamental de processos difusivos.

Por fim, a Figura 9 apresenta a posição média

$$\langle x(t) \rangle$$

em função do tempo. Teoricamente, devido à simetria da distribuição dos passos,

$$\langle x(t) \rangle = 0$$

para qualquer instante de tempo.

Os valores numéricos apresentam pequenas oscilações acima de zero, que são consequência do número finito de trajetórias utilizadas na estimativa. Essas flutuações não indicam a existência de uma direção preferencial no passeio. À medida que o número de realizações aumenta, espera-se que a média calculada se aproxime cada vez mais do valor teórico nulo.

Os resultados, portanto, confirmam as principais propriedades esperadas para o passeio aleatório unidimensional: a posição média permanece aproximadamente nula,

$$\langle x \rangle \approx 0,$$

enquanto o deslocamento quadrático médio cresce linearmente com o tempo,

$$\langle x^2 \rangle \propto t.$$

Além disso, as distribuições de posição tornam-se progressivamente mais largas e aproximadamente Gaussianas, evidenciando o comportamento difusivo do sistema.

Saída:

```
k necessario =      885553
precisao =    0.100000000000000001
```

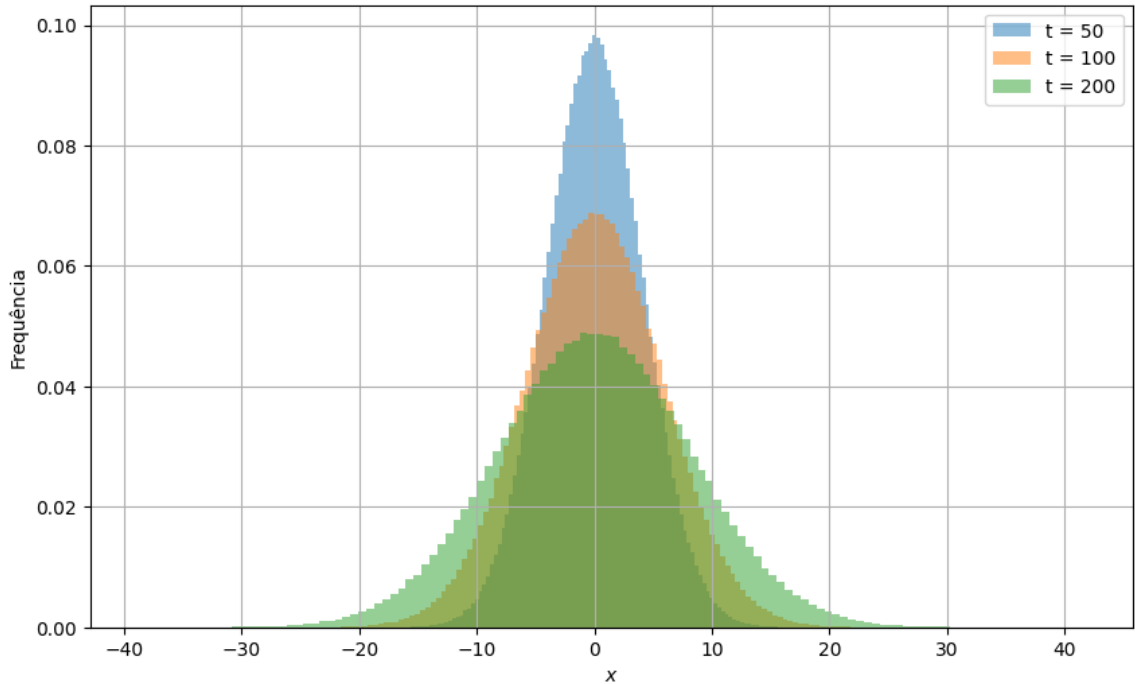


Figura 7: Distribuição das posições das partículas em diferentes instantes do passeio aleatório ($t = 50$, $t = 100$ e $t = 200$). À medida que o tempo aumenta, observa-se o alargamento da distribuição devido ao acúmulo dos deslocamentos aleatórios.

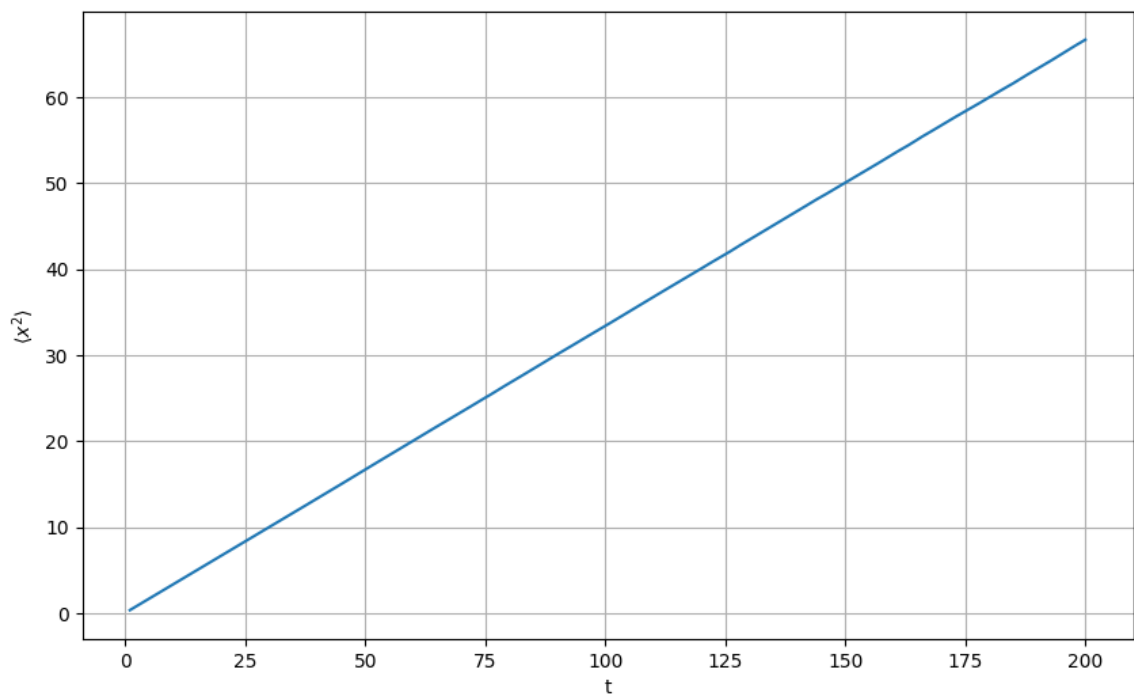


Figura 8: Evolução temporal do deslocamento quadrático médio $\langle x^2 \rangle$ em um passeio aleatório. O crescimento aproximadamente linear caracteriza o comportamento difusivo do processo.

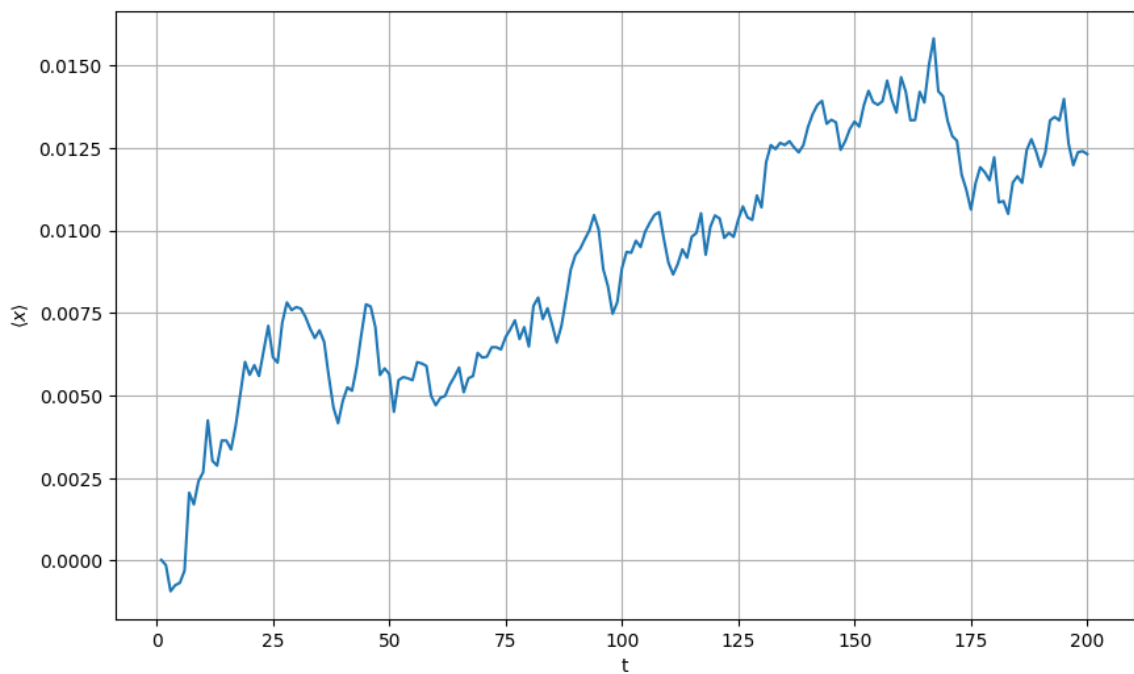


Figura 9: Evolução temporal da posição média $\langle x \rangle$ no passeio aleatório. As flutuações observadas em torno de zero são de origem ruidosa, sendo o valor teórico esperado $\langle x \rangle = 0$.

3.2 (b)

3.2.1 Descrição do Problema

Neste item, considera-se um passeio aleatório em duas dimensões.

A partícula parte da origem e seus deslocamentos obedecem às transformações

$$x \rightarrow x + \Delta x$$

e

$$y \rightarrow y + \Delta y,$$

onde

$$-1 \leq \Delta x \leq 1, \quad -1 \leq \Delta y \leq 1.$$

O objetivo é calcular o deslocamento quadrático médio

$$\langle r^2 \rangle = \langle x^2 + y^2 \rangle$$

em função do tempo para

$$t \leq 200.$$

A simulação deve ser realizada para diferentes números de passeios k , determinando-se o valor a partir do qual a curva

$$\langle r^2 \rangle \times t$$

não apresenta mais alterações superiores a aproximadamente 10%.

3.2.2 Estratégia de Solução

A simulação foi realizada gerando um conjunto de trajetórias independentes em duas dimensões. Cada trajetória inicia-se na origem,

$$x(0) = 0, \quad y(0) = 0,$$

e, a cada passo temporal, as duas coordenadas são atualizadas independentemente segundo

$$x(t+1) = x(t) + \Delta x$$

e

$$y(t+1) = y(t) + \Delta y.$$

Os deslocamentos Δx e Δy são gerados pela função ‘delta()’, que transforma um número aleatório uniforme no intervalo $[0, 1)$ em um número uniformemente distribuído entre -1 e 1 por meio da relação

$$\Delta = 2r - 1.$$

Para cada trajetória e para cada instante de tempo, calcula-se a distância quadrática em relação à origem,

$$r^2(t) = x^2(t) + y^2(t).$$

O programa acumula os valores de r^2 para todas as trajetórias geradas, permitindo calcular posteriormente o deslocamento quadrático médio,

$$\langle r^2(t) \rangle = \frac{1}{k} \sum_{i=1}^k r_i^2(t).$$

Também é acumulada a quantidade

$$r^4(t) = [r^2(t)]^2,$$

necessária para calcular o desvio padrão das realizações de r^2 . Dessa forma, o desvio padrão é estimado por

$$\sigma_{r^2}(t) = \sqrt{\langle r^4(t) \rangle - \langle r^2(t) \rangle^2}.$$

O erro estatístico da média é então calculado utilizando

$$\Delta \langle r^2(t) \rangle = \frac{\sigma_{r^2}(t)}{\sqrt{k}}.$$

Foi adotado como critério de convergência o valor

$$\epsilon = 10^{-1}.$$

Não é correto interpretar o enunciado como erro relativo, pois o erro relativo da média da posição não pode ser computado, pois $\langle x \rangle = 0$. Portanto, interpretamos o valor do erro como erro absoluto.

Após a geração de cada nova trajetória, o programa verifica o erro da estimativa de

$$\langle r^2(t) \rangle$$

para todos os instantes entre $t = 1$ e $t = 200$. O menor número de passeios para o qual todos os erros se tornam inferiores à tolerância estabelecida é identificado como

$$k_{\min}.$$

Do ponto de vista teórico, como os deslocamentos nas direções x e y são independentes e identicamente distribuídos, tem-se

$$\langle \Delta x \rangle = \langle \Delta y \rangle = 0.$$

Além disso, para uma variável aleatória uniformemente distribuída no intervalo entre -1 e 1 ,

$$\langle (\Delta x)^2 \rangle = \langle (\Delta y)^2 \rangle = \frac{1}{3}.$$

Após t passos independentes, espera-se portanto que

$$\langle x^2(t) \rangle = \frac{t}{3}$$

e

$$\langle y^2(t) \rangle = \frac{t}{3}.$$

Consequentemente, o deslocamento quadrático médio em duas dimensões deve obedecer à relação

$$\langle r^2(t) \rangle = \langle x^2(t) \rangle + \langle y^2(t) \rangle = \frac{2t}{3}.$$

Assim, espera-se que o gráfico de

$$\langle r^2 \rangle$$

em função de t apresente um crescimento aproximadamente linear, característica do regime de difusão associado ao passeio aleatório.

```

1
2  program tarefa3b
3
4  integer*4 k, k_total, t, k_min
5  integer*4 precisao_ok
6
7  real*8 delta
8  real*8 x, y, r2
9
10 real*8 soma_r2(200)
11 real*8 soma_r4(200)
12
13 real*8 media_r2
14 real*8 media_r4
15
16 real*8 desvio_r2
17
18 real*8 erro_r2
19
20 real*8 epsilon
21 parameter(epsilon = (10d0)**(-1))
22
23 ! Inicializa os acumuladores
24 do t = 1, 200
25     soma_r2(t) = 0d0
26     soma_r4(t) = 0d0
27 end do
28
29 k_total = 10**7
30 k_min = 0
31
32 ! Gera k caminhos
33 do k = 1, k_total
34
35     x = 0d0
36     y = 0d0
37

```

```

38      ! Cada caminho com 200 passos
39      do t = 1, 200
40
41          x = x + delta() ! coordenada horizontal
42          y = y + delta() ! coordenada vertical
43          r2 = x**2 + y**2 ! distância quadrática
44
45          soma_r2(t) = soma_r2(t) + r2
46          soma_r4(t) = soma_r4(t) + r2**2 ! para cálculo do desvio padrão
47
48      end do
49
50      ! A cada caminho gerado, verifica a precisao para todos os tempos
51      if (k .ge. 2) then
52
53          precisao_ok = 1
54
55          do t = 1, 200
56
57              media_r2 = soma_r2(t)/dble(k)
58              media_r4 = soma_r4(t)/dble(k)
59
60              ! Desvio padrao da média de r^2
61              desvio_r2 = dsqrt(media_r4 - media_r2**2)
62
63              ! Erro da média de r^2
64              erro_r2 = desvio_r2/dsqrt(dble(k))
65
66              ! Se qualquer tempo nao atingir a precisao,
67              ! este k ainda nao e suficiente
68
69              if (erro_r2 .gt. epsilon) then
70                  precisao_ok = 0
71              end if
72
73          end do
74
75          ! Se todos os tempos atingiram a precisao,
76          ! encontramos o k necessario
77          if (precisao_ok .eq. 1) then
78              k_min = k
79              goto 100
80          end if
81
82      end if
83
84  end do
85
86  100  continue
87
88  print *, 'k necessario = ', k_min

```

```

89  print *, 'precisao = ', epsilon
90
91  ! Abrir arquivos
92  open(unit=10, file='tarefa-3b-saida-1.dat')
93
94  ! Médias de r^2
95  do t = 1, 200
96      write(10, *) t, soma_r2(t)/dble(k_min)
97  end do
98
99  close(10)
100
101  end
102
103  function delta()
104      ! Gera um numero aleatorio uniformemente distribuido entre -1 e 1.
105      ! Entrada: Nenhuma.
106      ! Saida: delta - numero aleatorio gerado no intervalo.
107
108      real*8 delta
109
110      delta = 2d0 * rand() - 1d0 ! bijeção linear entre [0,1) e [-1,1]
111
112      return
113  end

```

3.2.3 Resultados Obtidos

O critério de precisão adotado foi satisfeito para

$$k_{\min} = 1775550.$$

Portanto, foram necessários aproximadamente

$$1.78 \times 10^6$$

passeios independentes para que o erro estatístico absoluto da estimativa de

$$\langle r^2(t) \rangle$$

fosse inferior a

$$\epsilon = 0.1$$

em todos os instantes de tempo considerados.

O número de trajetórias necessário é maior do que no caso unidimensional. Isso ocorre porque a quantidade analisada neste item,

$$r^2 = x^2 + y^2,$$

é formada pelas contribuições estocásticas de duas coordenadas independentes, o que aumenta a sua variância. Consequentemente, é necessário um número elevado de realizações para que o erro da média seja reduzido.

A Figura 10 apresenta a evolução do deslocamento quadrático médio em função do tempo. Observa-se que

$$\langle r^2 \rangle$$

aumenta aproximadamente de forma linear com t , conforme esperado para um processo difusivo.

Pela linearidade do valor esperado, como cada coordenada contribui com

$$\langle x^2(t) \rangle = \frac{t}{3}$$

e

$$\langle y^2(t) \rangle = \frac{t}{3},$$

espera-se que

$$\langle r^2(t) \rangle = \frac{2t}{3}.$$

Assim, a inclinação esperada para a curva é

$$\frac{2}{3}.$$

O comportamento aproximadamente linear observado no resultado numérico está de acordo com essa previsão. Pequenas diferenças em relação à curva teórica são atribuídas às flutuações estatísticas inerentes ao número finito de trajetórias utilizadas.

O crescimento linear de

$$\langle r^2 \rangle$$

com o tempo caracteriza o regime difusivo. Em particular,

$$\langle r^2(t) \rangle \propto t,$$

de modo que a distância típica percorrida pela partícula cresce como

$$r_{\text{típico}} \sim \sqrt{t}.$$

Esse resultado mostra que, embora a partícula realize um número de passos proporcional ao tempo, seu afastamento típico em relação à origem cresce apenas com a raiz quadrada do número de passos, devido ao caráter aleatório e à ausência de uma direção preferencial no movimento.

A comparação das curvas obtidas para diferentes valores de k permite verificar a convergência da simulação. À medida que o número de passeios aumenta, as flutuações estatísticas da curva diminuem e a estimativa de

$$\langle r^2(t) \rangle$$

torna-se progressivamente mais estável. Para

$$k = k_{\min} = 1775550,$$

a variação estatística encontra-se dentro da tolerância adotada, indicando que esse número de trajetórias é suficiente para a precisão estabelecida no problema.

Saída:

```
k necessario =      1775550
precisao =    0.100000000000000001
```

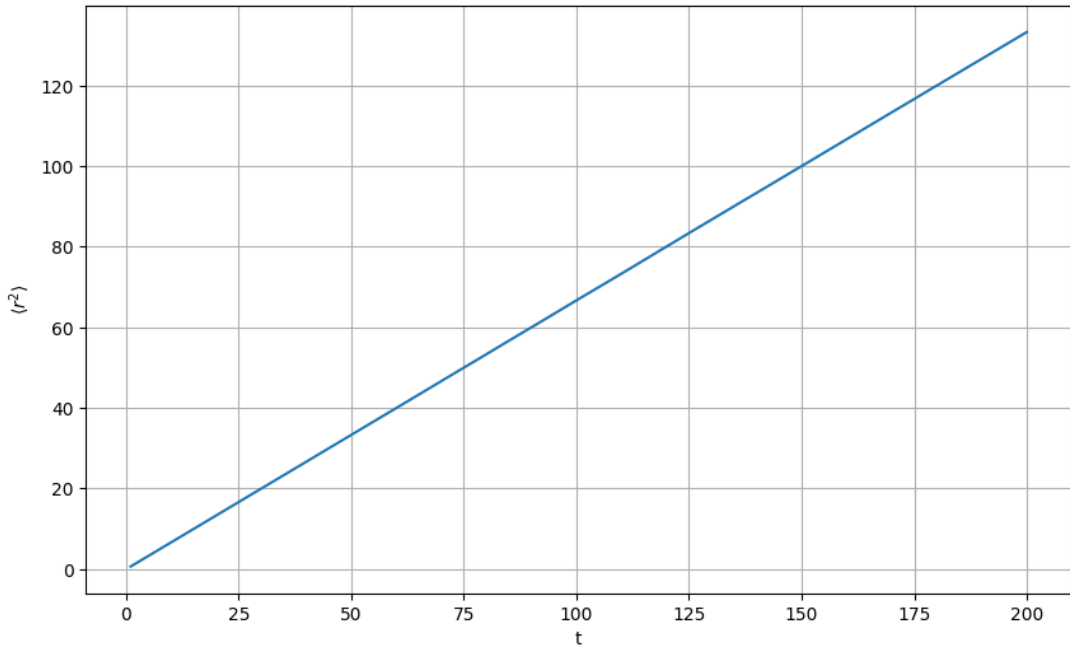


Figura 10: Deslocamento quadrático médio $\langle r^2 \rangle$ em função do tempo para o valor encontrado necessário para precisão $k = k_{min}$. O crescimento aproximadamente linear evidencia o comportamento difusivo do passeio aleatório em duas dimensões.

3.3 (c) – Opcional

3.3.1 Descrição do Problema

Neste item, considera-se um passeio aleatório em duas dimensões delimitado por um recipiente de tamanho característico L .

Caso a partícula ultrapasse uma das fronteiras do recipiente, ela deve sofrer reflexão especular e retornar ao interior do sistema.

O problema é dividido em três partes:

1. Construir a curva

$$\langle r^2 \rangle \times t$$

para

$$t \leq 10^4$$

e

$$k = 10^4$$

para diferentes valores de L ;

2. Simular

$$N = 10^3$$

partículas inicialmente concentradas na origem de um recipiente quadrado e registrar suas posições para diferentes instantes de tempo;

3. Calcular a entropia de Gibbs

$$S = - \sum_i p_i \ln p_i,$$

onde p_i representa a probabilidade de encontrar uma partícula na célula i de uma partição do recipiente.

Os resultados devem ser discutidos em relação à evolução para uma distribuição espacial aproximadamente uniforme e à noção de seta do tempo.

3.3.2 Estratégia de Solução

3.3.3 Resultados Obtidos

Figura 11: Deslocamento quadrático médio $\langle r^2 \rangle$ em função do tempo para diferentes tamanhos do recipiente.

Figura 12: Distribuição espacial das partículas para diferentes instantes de tempo.

Figura 13: Entropia de Gibbs em função do tempo para diferentes números de partições do recipiente.